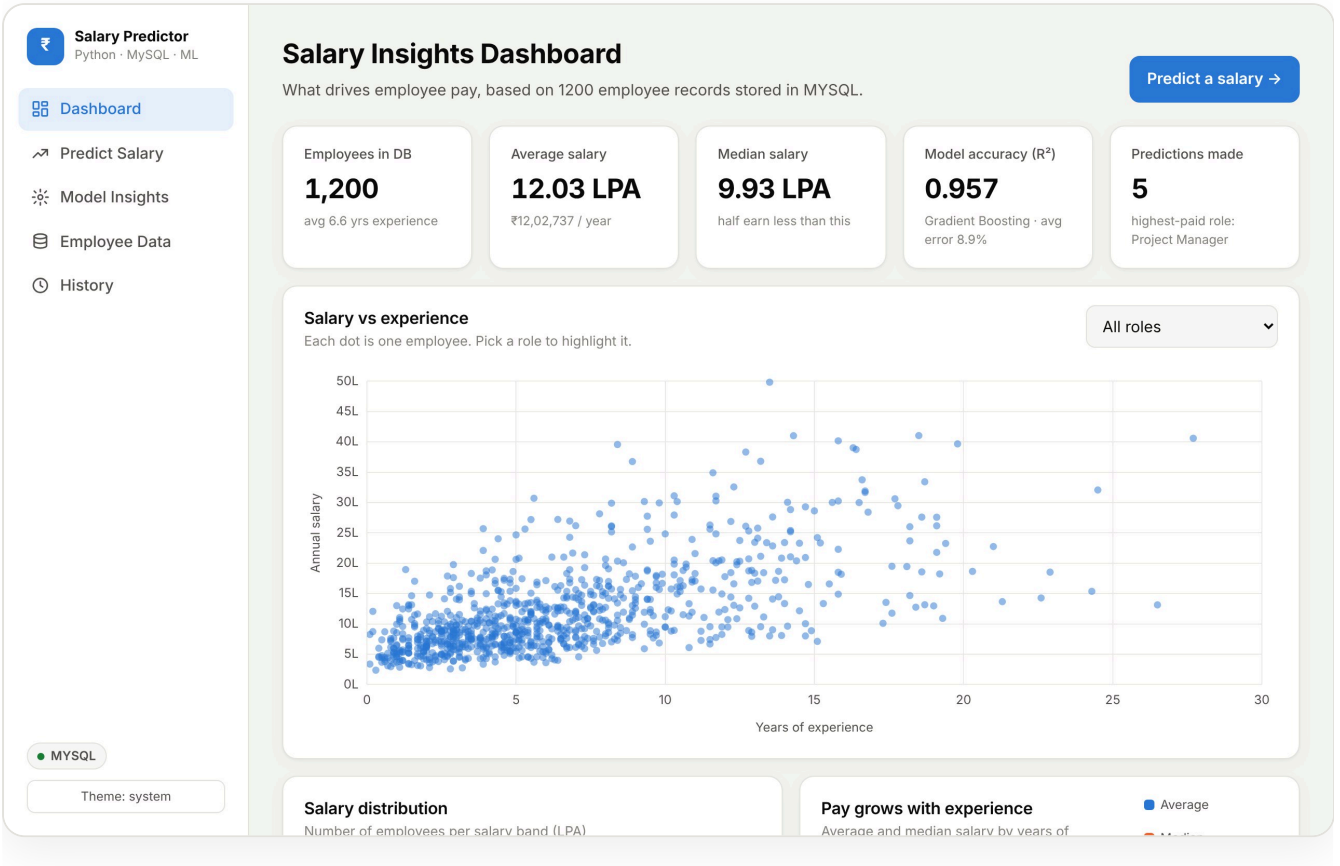
PYTHON + MySQL · AIML MINI PROJECT · No. 12

Employee Salary Prediction System

Project report, system explanation and viva question bank

Python 3 | MySQL | scikit-learn | Regression | Gradient Boosting | Flask | Chart.js | pandas



Submitted by

Roll number

Department / Branch

Academic year

Project guide

College

Contents

1	Abstract	3
2	Introduction & Problem Statement	4
3	Requirements	6
4	System Architecture	8
5	Database Design (MySQL)	10
6	Dataset & Features	12
7	Machine Learning Methodology	15
8	Results & Model Evaluation	17
9	Version 1 – Command-Line Application	19
10	Version 2 – Web Application	23
11	Installation & Running	33
12	Testing	34
13	Advantages, Limitations & Future Scope	35
14	Conclusion	36
15	Viva Questions & Answers (100+)	37
16	Quick Revision Sheet	49

1 Abstract

The **Employee Salary Prediction System** estimates the annual salary (CTC, in Indian rupees) of an employee from their professional profile. It is built with **Python**, stores all data in a **MySQL** database and uses supervised **regression** to learn how salary depends on experience, education, skills, job role, city tier, company type, certifications and performance rating.

Employee records are stored in MySQL in normalised tables, with skills kept in a many-to-many relationship. The training program reads those records with a SQL `JOIN`, turns them into numeric features, and trains five regression models: Linear, Ridge, Polynomial, Random Forest and Gradient Boosting. Every model is tested on 20% of the data it never saw and checked with 5-fold cross-validation. The model with the best cross-validated R^2 is saved and used for predictions. On the sample data of 1,200 employees, **Gradient Boosting** wins with **R^2 = 0.957** and an average error of about **8.9%**.

The project comes in **two versions** that share the same database and model:

- **Version 1, command line:** a menu-driven program in the terminal with text-based bar charts.
- **Version 2, web application:** a Flask website with an analytics dashboard, a prediction page with in-depth visualisation, model insights, employee record management and prediction history, using more than 20 interactive charts.

Every prediction is shown with an **80% likely salary range**, a **percentile** among employees in the same role, a **salary growth projection**, and **what-if analysis**. The what-if analysis shows how much each skill, degree, city or company type would change the salary. Every prediction is saved back to MySQL.

0.957	8.9%	5	5
R^2 score on unseen data	Mean absolute % error	Regression models compared	MySQL tables

Keywords: salary prediction, supervised learning, regression, gradient boosting, MySQL, Flask, feature engineering, cross-validation, explainable ML.

2 Introduction & Problem Statement

2.1 Background

Deciding a fair salary is one of the hardest jobs for HR teams and job seekers. Pay depends on many factors at once: years of experience, qualification, technical skills, role, city and the type of company. Doing it by hand or with a spreadsheet is slow, inconsistent and easily biased. Organisations already store employee data in databases, so a machine-learning model can learn the pattern from that history and give a quick, consistent and explainable estimate.

2.2 Problem statement

Given an employee's details (experience, education, skills, job role, location, company type, certifications and performance rating), predict their expected annual salary, and explain which factors drive it, using historical employee data stored in a MySQL database.

2.3 Objectives

1. Design a normalised **MySQL database** to store employees, their skills, predictions and model training logs.
2. Build a **regression model** that predicts salary accurately (target R^2 above 0.9).
3. **Compare several algorithms** fairly and choose the best one automatically.
4. Give the **input screen** (employee details) and **output screen** (predicted salary) in two interfaces: CLI and Web.
5. Make predictions **explainable**: likely range, peer comparison, feature importance and what-if analysis.
6. Let users **add new records** and **retrain** the model so it improves over time.

2.4 Existing system vs proposed system

Aspect	Existing (manual / spreadsheet)	Proposed system
Method	Gut feeling, salary surveys, fixed pay bands	Machine learning trained on historical data
Speed	Slow; each case reviewed by hand	Instant prediction
Consistency	Different people give different numbers	Same input always gives the same output
Factors considered	Usually 2–3 (experience, degree)	14 features including 10 individual skills
Explanation	None	Range, percentile, feature importance, what-if
Data storage	Scattered Excel files	Central MySQL database with history

Improvement	Static	Retrain whenever new records are added

2.5 Scope

The system targets the Indian IT and corporate job market: 8 job roles, 4 education levels, 3 city tiers, 3 company types and 10 skills. Salaries are in rupees per annum and are displayed in Indian format (₹12,34,567) and in LPA (lakhs per annum). It can be used by HR departments for offer planning, by students and job seekers to check their market value, and as a teaching example of a full ML + database pipeline.

3 Requirements

3.1 Hardware

Component	Minimum	Recommended
Processor	Dual core, 1.8 GHz	Quad core i5 / Ryzen 5
RAM	4 GB	8 GB
Disk	1 GB free	2 GB free
Display	1366 × 768	1920 × 1080 (the web UI also works on phones)

3.2 Software

Software	Version	Purpose
Windows / Linux / macOS	Windows 10+	Operating system
Python	3.9 or newer	Programming language
MySQL Server / XAMPP	MySQL 8 / MariaDB 10+	Relational database
numpy, pandas	latest	Numerical arrays and data frames
scikit-learn	1.4+	Machine learning algorithms and evaluation
joblib	1.3+	Saving and loading the trained model
Flask	3.0+	Web framework for Version 2
mysql-connector-python	8.3+	Python ↔ MySQL driver
Chart.js	4.4 (bundled)	Interactive charts in the browser
Web browser	Chrome / Edge / Firefox	To open the web version

3.3 Functional requirements

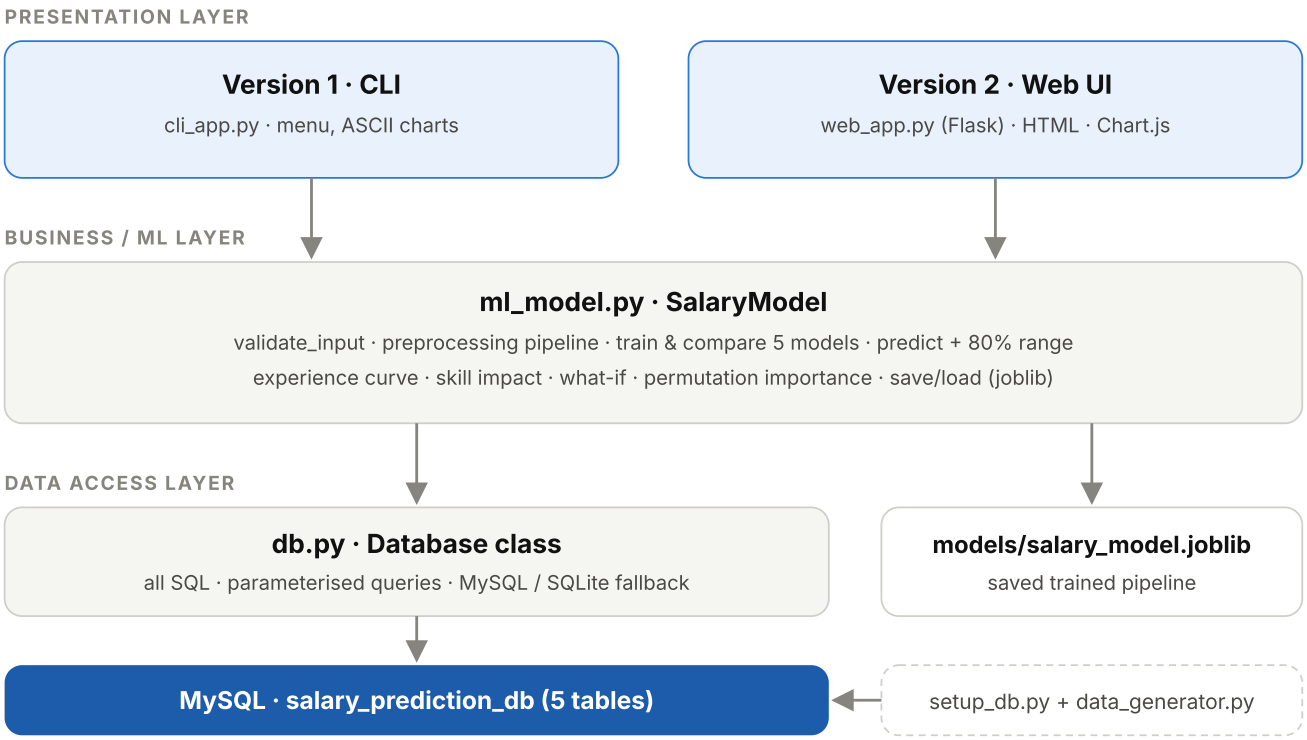
- Accept employee details as input and display the predicted salary.
- Store employee records, skills and predictions in MySQL.
- Train, evaluate and compare regression models, and retrain on demand.
- Add, search, delete and export (CSV) employee records.
- Visualise the dataset, the model's accuracy and each prediction.
- Validate every input (ranges, allowed options) and show clear error messages.

3.4 Non-functional requirements

- **Performance:** a prediction returns in well under one second; training takes about 10 seconds.
- **Usability:** one-click Windows installer, simple menus and a clean web UI with light and dark themes.
- **Reliability:** falls back to SQLite automatically if MySQL is not running, so a demo never fails.
- **Security:** all SQL uses parameterised queries, which prevents SQL injection.
- **Portability:** runs on Windows, Linux and macOS, and the web UI works offline.

4 System Architecture

The project uses a **layered architecture**. Both user interfaces sit on top of one shared ML core and one shared database layer, so there is no duplicated logic, and a prediction made in the CLI appears in the web history.



4.1 Module description

File / module	Responsibility
`config.py`	All settings in one place: database login (from `db_settings.ini` or environment variables), feature lists, allowed options, ₹/LPA formatting and the Windows console fix.
`schema.sql`	MySQL schema: CREATE DATABASE plus 5 tables with primary keys, foreign keys and constraints.
`db.py`	Database layer. Connects to MySQL (or falls back to SQLite), creates tables, inserts and fetches employees with their skills, saves predictions and model runs. No SQL exists anywhere else.
`data_generator.py`	Generates 1,200 realistic employee records using known market effects plus random noise.
`ml_model.py`	Core ML: input validation, preprocessing pipeline, training and comparing 5 regressors, evaluation metrics, feature importance, prediction with a range, and what-if functions.
`setup_db.py`	One-time setup: create tables, seed data and train the first model. `--reset` rebuilds everything.

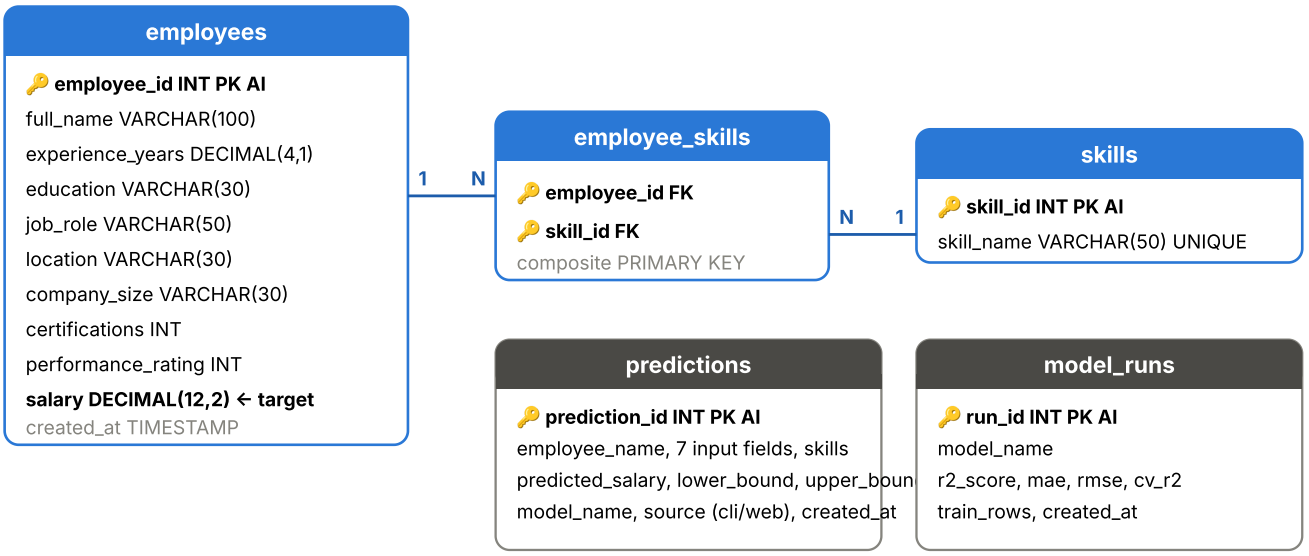
`configure_db.py`	Asks for the MySQL login, tests the connection and saves `db_settings.ini`.
`cli_app.py`	**Version 1**: interactive menu and one-line commands with ASCII bar charts and histograms.
`web_app.py` + `templates/` + `static/`	**Version 2**: Flask routes, Jinja2 HTML pages, CSS and Chart.js visualisations.
`START_HERE.bat`	One-click Windows installer and launcher.

4.2 Data flow of a single prediction

1. The user enters details in the CLI prompts or the web form.
2. `validate_input()` checks ranges and allowed values and normalises the skills list.
3. `to_frame()` converts the input into the same 17 feature columns used in training (10 skills become 0/1 columns).
4. The saved pipeline scales and one-hot encodes the features, and the Gradient Boosting model predicts *log(salary)*, which is converted back with e^x.
5. The 80% range is computed, and peers are fetched from MySQL to calculate the percentile.
6. The what-if functions run the model again on changed copies of the input (more experience, an added skill, a different degree, and so on).
7. The result is displayed as text and charts and inserted into the `predictions` table.

5 Database Design (MySQL)

The database `salary_prediction_db` is normalised to **Third Normal Form (3NF)**. An employee can have many skills and a skill belongs to many employees, so skills are kept in their own table and linked with a junction table. Storing them as a comma-separated text column would break First Normal Form.



5.1 Table descriptions

Table	Rows (sample)	Purpose
`employees`	1,200	Historical employee records used to train the model. `salary` is the target variable (label).
`skills`	10	Master list of skills: Python, Java, SQL, JavaScript, Machine Learning, Cloud, Docker, Excel, Communication, Leadership.
`employee_skills`	4,279	Junction table for the many-to-many relationship. Composite primary key (employee_id, skill_id) and `ON DELETE CASCADE`.
`predictions`	grows	Audit log of every prediction from the CLI and the web: inputs, output, range, model name and source.
`model_runs`	grows	Evaluation metrics of every training run, to track model quality over time.

5.2 Key SQL used

Loading training data: one query joins the three tables and collects each employee's skills into a single string:

```
SELECT e.employee_id, e.full_name, e.experience_years, e.education, e.job_role,
       e.location, e.company_size, e.certifications, e.performance_rating, e.salary,
       GROUP_CONCAT(s.skill_name) AS skills
FROM employees e
LEFT JOIN employee_skills es ON es.employee_id = e.employee_id
LEFT JOIN skills s           ON s.skill_id     = es.skill_id
GROUP BY e.employee_id, ... ORDER BY e.employee_id DESC;
```

Saving a prediction uses a parameterised query. Values are sent separately from the SQL text, so input like `' OR 1=1 --` cannot change the query:

```
INSERT INTO predictions (employee_name, experience_years, education, job_role, ...,
                         predicted_salary, lower_bound, upper_bound, model_name, source)
VALUES (%s, %s, %s, %s, ..., %s, %s, %s, %s, %s)
```

Why a LEFT JOIN?

An employee with no skills has no rows in `employee_skills`. An INNER JOIN would drop that employee completely, while a LEFT JOIN keeps them with `skills = NULL`. The code then treats NULL as an empty skill list.

6 Dataset & Features

Real company salary data is confidential, so the project generates a **realistic synthetic dataset** of **1,200 employees** (`data_generator.py`). Each salary is built from well-known market effects plus random noise. The ML model **never sees this formula**: it only sees the rows in MySQL and has to learn the relationships, the same way it would with real data.

1,200	12.03 L	9.93 L	2.4 – 51 L
Employees	Average salary	Median salary	Salary range (LPA)

6.1 Features (inputs) and target (output)

Feature	Type	Values	Pre-processing
experience_years	Numeric	0 – 28 years	StandardScaler
certifications	Numeric	0 – 6	StandardScaler
performance_rating	Numeric (ordinal)	1 – 5	StandardScaler
education	Categorical	Diploma, Bachelor's, Master's, PhD	One-hot encoding
job_role	Categorical	8 roles	One-hot encoding
location	Categorical	Tier-1 / Tier-2 / Tier-3 city	One-hot encoding
company_size	Categorical	Startup, Mid-size, MNC	One-hot encoding
10 skill flags	Binary	0 / 1 each	Passed through as they are
salary (target)	**Continuous**	**₹ per year**	**log transform (model learns log salary)**

After preprocessing there are **31 numeric columns**: 3 scaled numbers, 18 one-hot columns and 10 skill flags.

6.2 How the sample data was generated

Salary = role base pay × experience growth curve × education factor × city factor × company factor × skill premiums × certification bonus × rating bonus × random noise.

Factor	Effect used in generation
Role base (fresher)	₹2.8 L (Sales) up to ₹6.5 L (Project Manager)
Experience	8–14% growth per year depending on role, flattening after about 20 years
Education	Diploma ×0.82 · Bachelor's ×1.00 · Master's ×1.18 · PhD ×1.38

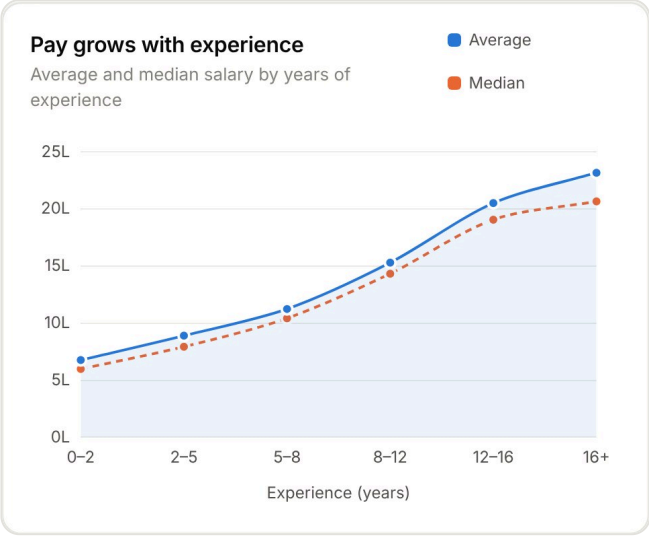
City	Tier-1 ×1.18 · Tier-2 ×1.00 · Tier-3 ×0.84
Company	Startup ×0.94 · Mid-size ×1.00 · MNC ×1.17
Skills	Machine Learning +14%, Cloud +10%, Python/Docker +6%, Java +5% … (Leadership grows with seniority)
Certifications / rating	+2.5% per certification; ±4.5% per rating point above or below 3
Noise	Log-normal, σ = 0.09 (about ±9%), standing in for factors we don't record, like negotiation

Skills are assigned with role-specific probabilities (for example, 85% of Data Scientists know ML, 90% of DevOps engineers know Docker), so the data looks like a real workforce.

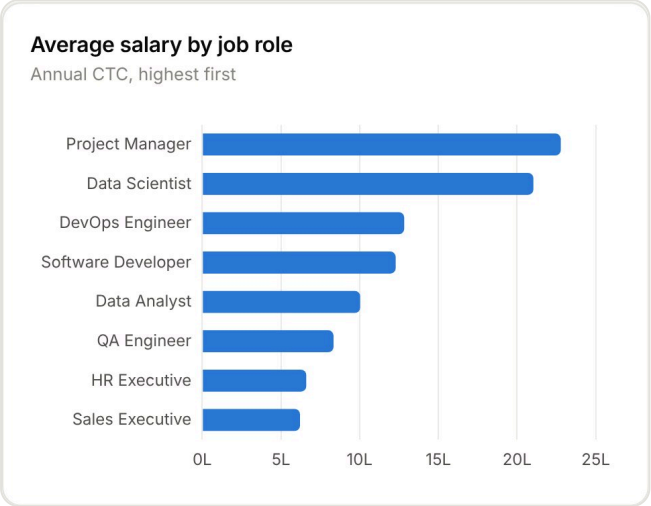
6.3 Exploratory data analysis



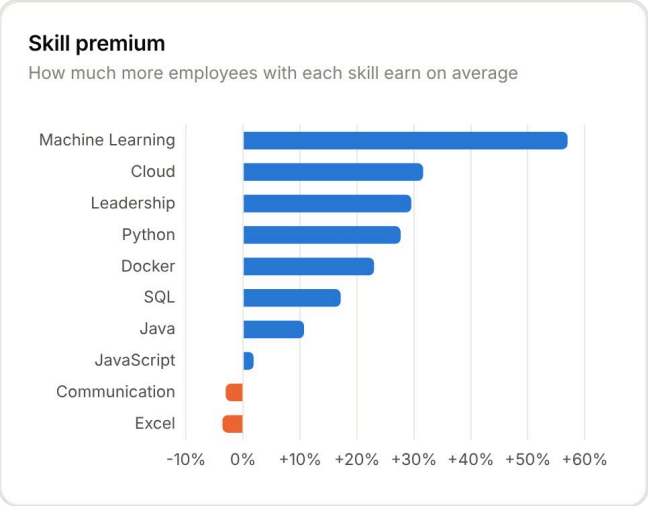
Salary distribution is right-skewed: many juniors, few very senior people. This is why the model learns log(salary).



Pay rises with experience, from 6.8 L (0–2 yrs) to 23 L (16+ yrs).



Average salary by role: Project Managers and Data Scientists earn the most.



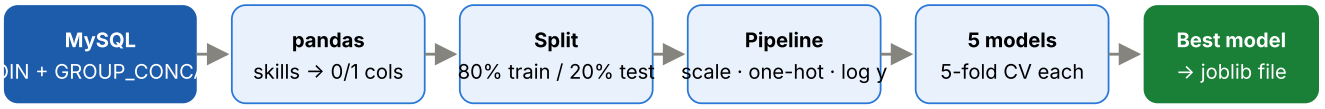
Skill premium: Machine Learning holders earn about 57% more on average. This is raw correlation, partly because of their role.

7 Machine Learning Methodology

7.1 Problem type

This is **supervised learning** because every training row has the correct answer (salary). It is a **regression** problem because the output is a continuous number, not a category.

7.2 Training pipeline



Then on the 20% test set: R^2, MAE, RMSE, MAPE · 80% prediction range · permutation feature importance · log to model_runs table

7.3 Pre-processing

- **One-hot encoding** turns each category into its own 0/1 column (for example *job_role_Data Scientist*). Models need numbers, and giving categories ranks like 1, 2, 3 would imply an order that doesn't exist. `handle_unknown="ignore"` stops an unseen category from crashing the model.
- **Standard scaling** ($z = (x - \text{mean}) / \text{std}$) puts numeric features on the same scale. This matters for the linear models. Trees don't need it, but it does no harm.
- **Log-transform of the target** uses `TransformedTargetRegressor(func=np.log, inverse_func=np.exp)`. Salaries grow by *percentages* (a Master's adds about 18%, not a fixed ₹ amount), and the distribution is right-skewed. In log space those effects become additive and the errors become more even across low and high salaries.
- Everything sits inside one **scikit-learn Pipeline**, so the exact same transformations are applied during training and prediction, and nothing from the test set leaks into training.

7.4 Algorithms compared

Model	How it works	Strength / weakness
Linear Regression	Fits $\hat{y} = b_0 + b_1x_1 + \ldots + b_nx_n$ by minimising the sum of squared errors (Ordinary Least Squares).	Simple and interpretable, but cannot bend to curves.
Ridge Regression	Linear regression plus an L2 penalty $\alpha \cdot \Sigma b^2$ that keeps coefficients small.	Reduces overfitting and handles correlated features.
Polynomial Regression	Adds experience, experience^2 and experience^3 and pairwise interaction terms (such as role × experience), then fits Ridge ($\alpha = 10$).	Captures curves and interactions, but has many features.

Random Forest	250 decision trees, each trained on a bootstrap sample with random feature subsets. The prediction is the *average* of the trees (bagging).	Robust and non-linear, but its averaging smooths out extremes.
Gradient Boosting ★	**400 shallow trees (depth 3) built *one after another*. Each new tree fits the remaining errors (residuals) of the trees before it, scaled by a learning rate of 0.05 and trained on 90% subsamples.**	**Usually the most accurate on tabular data. Selected.**

7.5 Evaluation strategy

- **Hold-out test:** `train_test_split(test_size=0.2, random_state=42)` gives 960 rows for training and 240 for testing. The test rows are never seen during training.
- **5-fold cross-validation** on the training part: the data is split into 5 parts, the model trains on 4 and validates on the 5th, and this repeats 5 times. The mean R^2 is used to **choose** the model. That is more reliable than a single split, and it keeps the test set purely for the final report.
- `random_state = 42` everywhere makes results **reproducible**.

7.6 Metrics

$$R^2 = 1 - \Sigma(y - \hat{y})^2 \,/\, \Sigma(y - \bar{y})^2$$

R^2 (coefficient of determination) is the share of the variation in salary that the model explains. 1.0 is perfect, and 0 means no better than always predicting the average.

$$\mathrm{MAE} = (1/n)\,\Sigma\,|y - \hat{y}| \qquad \mathrm{RMSE} = \sqrt{[(1/n)\,\Sigma\,(y - \hat{y})^2]} \qquad \mathrm{MAPE} = (100/n)\,\Sigma\,|y - \hat{y}| \,/\, y$$

MAE is the average miss in rupees. **RMSE** penalises big misses more. **MAPE** is the average miss in percent, which is fairer across low and high salaries.

7.7 Prediction range (80% interval)

A single number hides uncertainty. On the test set the program computes the ratio *actual ÷ predicted* for every employee. 80% of the ratios lie between the 10th percentile (0.879) and the 90th percentile (1.160). For a new prediction P, the likely range is **0.879 × P to 1.160 × P**. This is an empirical (residual-quantile) prediction interval.

7.8 Explainability

- **Permutation feature importance:** shuffle one feature's column in the test set and measure how much R^2 drops. A big drop means the model relies on that feature. It works for any model.
- **What-if analysis:** keep the profile fixed, change one input (add a skill, change the degree, city, company or rating, or set experience to 0–25 years), predict again and show the difference.
- **Peer comparison:** the prediction's percentile among real employees with the same role in MySQL.

8 Results & Model Evaluation

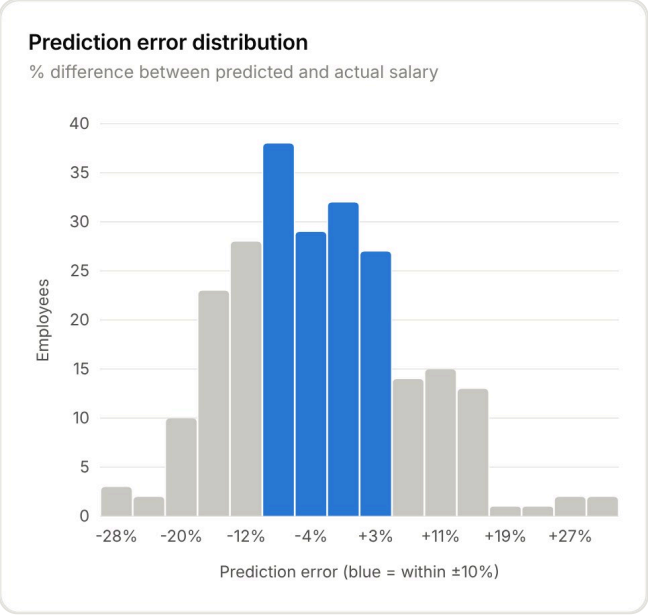
Model	R^2 (test)	CV R^2 (5-fold)	MAE	RMSE	MAPE
★ **Gradient Boosting**	**0.9566**	**0.9450 ± 0.008**	**₹1,05,930**	**₹1,46,861**	**8.93%**
Polynomial Regression	0.9437	0.9339 ± 0.025	₹1,12,003	₹1,67,314	9.21%
Linear Regression	0.9324	0.9139 ± 0.044	₹1,20,117	₹1,83,215	10.24%
Ridge Regression	0.9317	0.9130 ± 0.044	₹1,20,498	₹1,84,273	10.22%
Random Forest	0.8684	0.8525 ± 0.015	₹1,76,462	₹2,55,688	14.30%

8.1 Interpretation

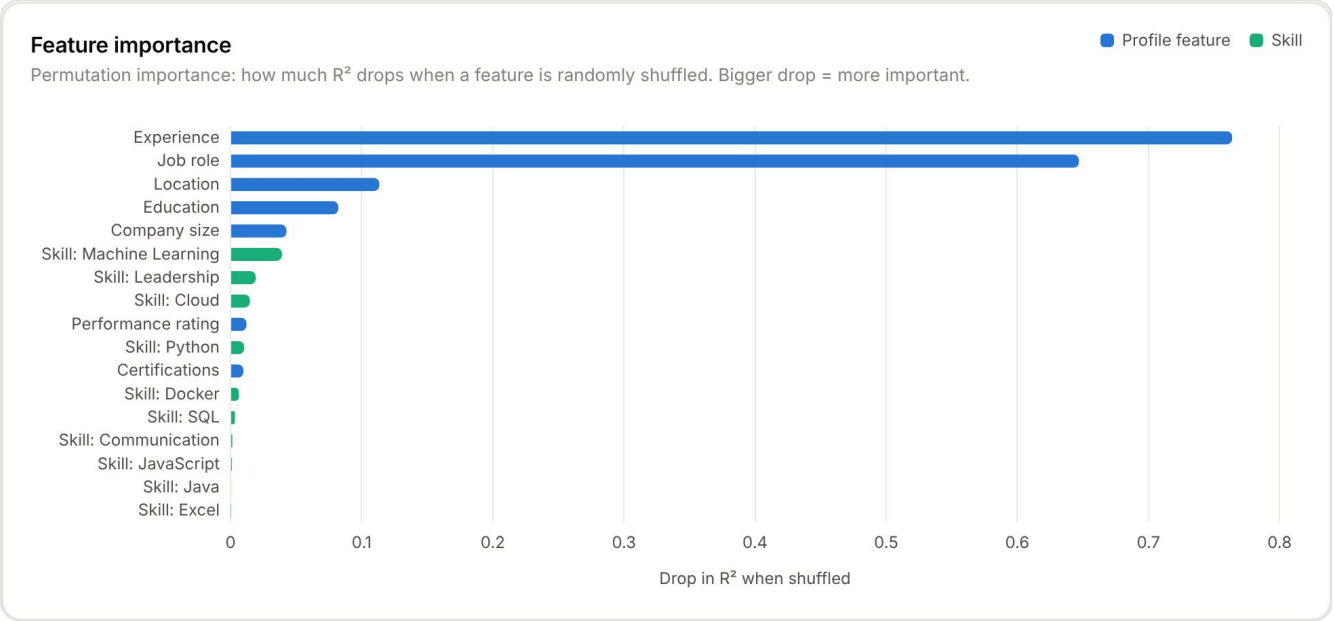
- **Gradient Boosting explains 95.7% of the variation in salary**, and its average error is about ₹1.06 L (8.9%). It also has the smallest CV spread (±0.008), so it is the most **stable** as well as the most accurate.
- Linear models already reach about 0.93 because of the log transform: multiplicative salary effects become additive in log space.
- **Random Forest** scores lowest. Averaging many deep trees flattens the extremes, and it under-predicts the few very senior, very high salaries.
- The remaining error is roughly the size of the ±9% random noise built into the data. That noise stands in for factors we do not record, such as negotiation skill, so no model could remove it.



Actual vs predicted: points hug the diagonal (perfect-prediction) line.



Error distribution is centred on 0%. The blue bars are within ±10%.



Feature importance: experience (0.76) and job role (0.65) dominate, followed by location (0.11), education (0.08) and company size (0.04). Machine Learning is the most valuable skill (0.04).

9 Version 1 – Command-Line Application

`cli_app.py` is a text-based program. It can run as an **interactive menu** (`python cli_app.py`) or as **one-line commands** (`predict` , `stats` , `model` , `train` , `history`) using `argparse` . Charts are drawn with Unicode block characters (█), so it works in any terminal.

Main menu

```
═══════════════════════════════════════════════════════════════
                  EMPLOYEE SALARY PREDICTION SYSTEM
═══════════════════════════════════════════════════════════════
              Python + MySQL · Regression   |   DB: MYSQL

   1. Predict salary for an employee
   2. Dataset summary & charts
   3. Model performance & feature importance
   4. Retrain model from MySQL data
   5. Add employee record
   6. Search employees
   7. Prediction history
   8. Database info
   0. Exit
```

Input: the program asks for each detail with a default value shown in brackets. Choices are entered by number, and skills are entered as comma-separated numbers (e.g. `1,3,5`). Every value is checked, and a wrong value is asked for again with an error message.

Output screen – prediction

```
═══════════════════════════════════════════════════════════════════
                    PREDICTED SALARY — Ravi Kumar
═══════════════════════════════════════════════════════════════════

  Annual CTC                  ₹23,11,904  (23.12 LPA)
  Monthly (gross)             ₹1,92,659
  Likely range (80%)          ₹20,31,483  –  ₹26,82,855
  Model used                  Gradient Boosting  (R² 0.957)

Input summary
─────────────────────────────────────────────────────────────────
  Data Scientist · 4.0 yrs · Master's · Tier-1 City · MNC
  Certifications: 2   Rating: ★★★★☆
  Skills: Python, SQL, Machine Learning

Compared with 116 Data Scientists in the database
─────────────────────────────────────────────────────────────────
  Role minimum          ▏                                7.87 LPA
  Role median           ▏▏▏▏▏▏▏▏▏▏▏▏                     18.85 LPA
  Your prediction       ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏                  23.12 LPA ◄ you
  Role maximum          ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  51.11 LPA

  You would earn more than 66% of Data Scientists.

Salary growth projection (same profile, more experience)
─────────────────────────────────────────────────────────────────
   0 yrs                ▏▏▏▏▏▏▏▏▏▏▏                      15.90 LPA
   2 yrs                ▏▏▏▏▏▏▏▏▏▏▏                      17.02 LPA
   4 yrs                ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏                  23.12 LPA ◄ you
   6 yrs                ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏                26.51 LPA
   8 yrs                ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏                28.87 LPA
  10 yrs                ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏           33.55 LPA
  12 yrs                ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏       37.98 LPA
  15 yrs                ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏   41.46 LPA
  20 yrs                ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  50.60 LPA
  25 yrs                ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  49.89 LPA

Skill impact — what each skill is worth for this profile
─────────────────────────────────────────────────────────────────
  ✓ Machine Learning    ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  +₹2,79,794
  + Cloud               ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏    +₹2,50,771
  ✓ Python              ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏      +₹2,27,066
  ✓ SQL                 ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  +₹95,854
  + Leadership          ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  +₹58,021
  + Communication       ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  +₹55,024
  + Docker              ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  +₹16,148
  + Excel               ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  +₹7,404
  + JavaScript          ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  +₹7,212
  + Java                ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  +₹1,271
  ✓ = skill you have (current contribution)   + = skill you could add (gain)

  Tip: learning Cloud (+10.8%), Leadership (+2.5%), Communication (+2.4%) gives the biggest raise.

Education what-if
─────────────────────────────────────────────────────────────────
  Diploma               ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  15.68 LPA
  Bachelor's            ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  20.30 LPA
  Master's              ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  23.12 LPA ◄ you
  PhD                   ▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏▏  25.15 LPA

  ✔ Prediction saved to the predictions table.
```

Dataset summary (option 2)

```
═══════════════════════════════════════════════════════════════
                        DATASET SUMMARY
═══════════════════════════════════════════════════════════════

  Records: 1200    Average: 12.03 LPA    Median: 9.93 LPA
  Min: 2.36 LPA    Max: 51.11 LPA    Avg experience: 6.6 yrs

Salary distribution
─────────────────────────────────────────────────────────────
     2.4 -   7.2 LPA ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ 326
     7.2 -  12.1 LPA ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓
```

Model performance (option 3)

```
═══════════════════════════════════════════════════════════════════════════
                          MODEL PERFORMANCE
═══════════════════════════════════════════════════════════════════════════
  Selected model : Gradient Boosting  (best cross-validated R²)
  Trained at     : 2026-10-12 05:44:27   on 960 rows, tested on 240 rows

Model comparison (test set)
──────────────────────────────────────────────────────────────────────────
  Model                  R²      CV R²           MAE         RMSE        MAPE
  ─────────────────────  ──────  ──────────────  ──────────  ──────────  ──────
  ★ Gradient Boosting    0.9566  0.9450 ±0.008  ₹1,05,930   ₹1,46,861   8.93%
    Polynomial Regression 0.9437  0.9339 ±0.025  ₹1,12,003   ₹1,67,314   9.21%
    Linear Regression    0.9324  0.9139 ±0.044  ₹1,20,117   ₹1,83,215   10.24%
    Ridge Regression     0.9317  0.9130 ±0.044  ₹1,20,498   ₹1,84,273   10.22%
    Random Forest        0.8684  0.8525 ±0.015  ₹1,76,462   ₹2,55,688   14.30%

  Gradient Boosting      ████████████████████████████ 0.9566 ◄ selected
  Polynomial Regression  ████████████████████████████ 0.9437
  Linear Regression      ███████████████████████████  0.9324
  Ridge Regression       ███████████████████████████  0.9317
  Random Forest          ████████████████████████     0.8684

Feature importance (drop in R² when the feature is shuffled)
──────────────────────────────────────────────────────────────────────────
  Experience             ████████████████████████████ 0.7639
  Job role               ██████████████████████       0.6470
  Location               ████                         0.1135
  Education              ███                          0.0822
  Company size           ██                           0.0427
  Skill: Machine Learnin ██                           0.0392
  Skill: Leadership      ▌                            0.0193
  Skill: Cloud           ▌                            0.0148
  Performance rating     ▌                            0.0122
  Skill: Python                                       0.0104
  Certifications                                      0.0098
  Skill: Docker                                       0.0064
  Skill: SQL                                          0.0033
  Skill: Communication                                0.0013
  Skill: JavaScript                                   0.0010
  Skill: Java                                         0.0006
  Skill: Excel                                        0.0003

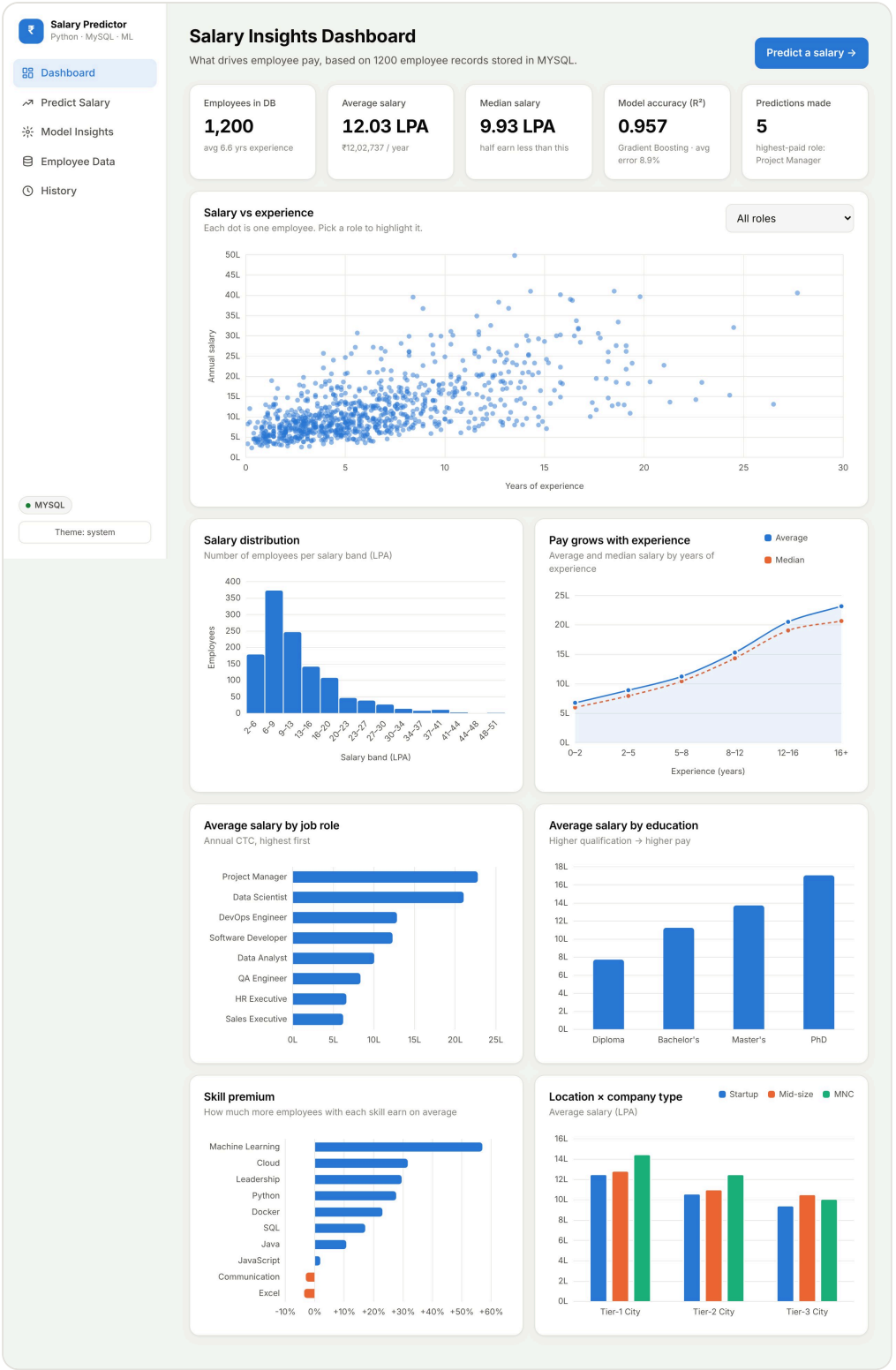
Sample predictions on unseen test employees
──────────────────────────────────────────────────────────────────────────
  Role                 Exp   Actual        Predicted     Error
  ───────────────────  ────  ────────────  ────────────  ──────
  Software Developer   2.0   ₹6,62,800     ₹6,77,561     +2.2%
  QA Engineer          6.5   ₹7,66,800     ₹7,78,022     +1.5%
  HR Executive         5.6   ₹3,68,200     ₹4,33,822     +17.8%
  Software Developer   5.5   ₹7,09,700     ₹7,77,019     +9.5%
  Software Developer   6.7   ₹14,51,200    ₹16,47,368    +13.5%
  DevOps Engineer      4.6   ₹13,45,600    ₹11,10,346    -17.5%
  Sales Executive      3.8   ₹4,27,100     ₹5,23,836     +22.6%
  Software Developer   2.3   ₹6,68,700     ₹5,67,705     -15.1%
  Software Developer   4.2   ₹12,43,100    ₹10,71,278    -13.8%
  Data Analyst         6.0   ₹12,83,500    ₹10,49,116    -18.3%
```

10 Version 2 – Web Application

`web_app.py` is a **Flask** application. Each page is a **route** that queries MySQL through `db.py`, calls the ML core and renders a **Jinja2** HTML template. Data for the charts is passed to the browser as JSON, and **Chart.js** draws interactive charts with hover tooltips. The site has a sidebar menu, light and dark themes and a phone-friendly layout.

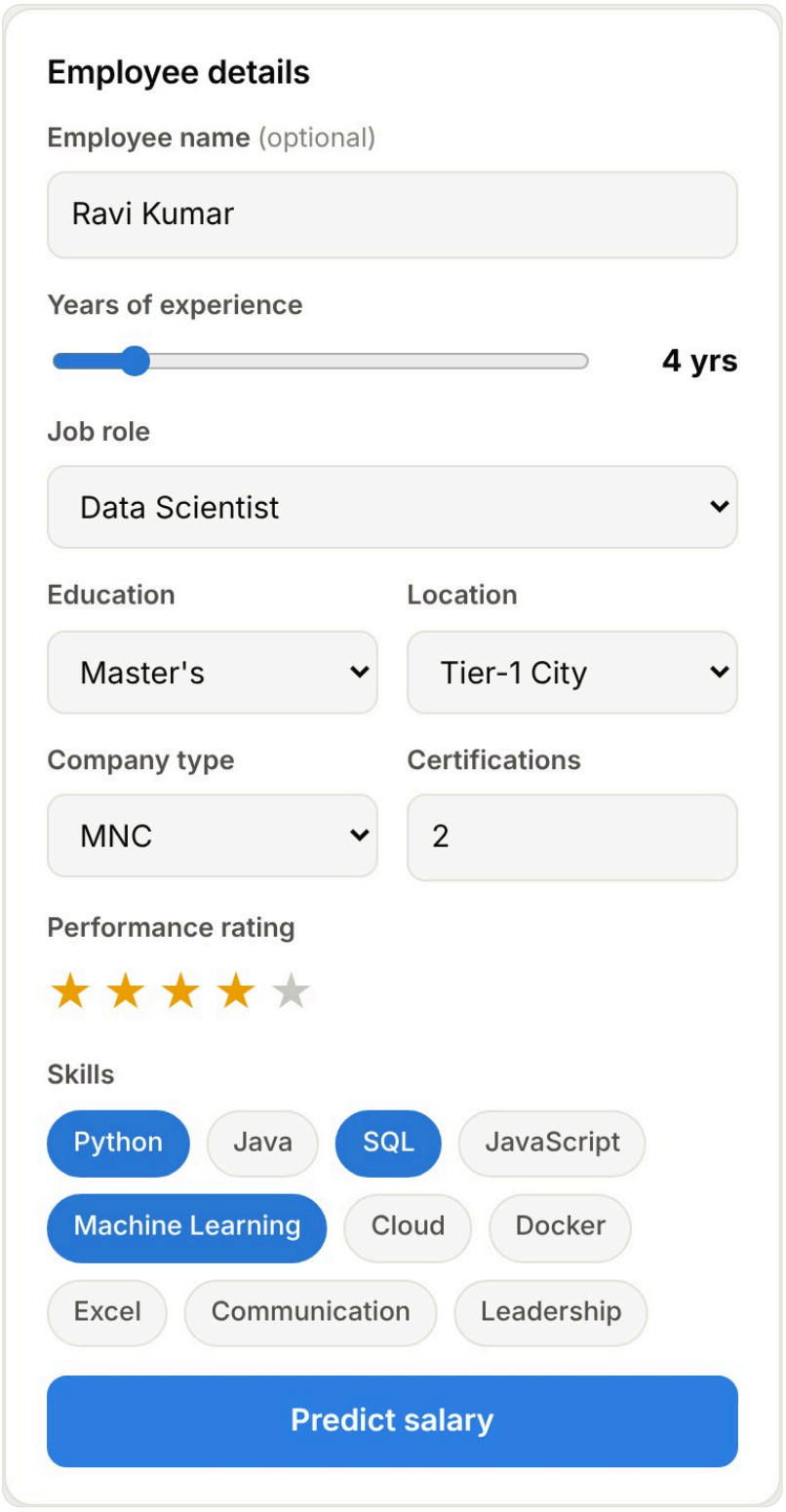
Route	Method	Purpose
`/`	GET	Dashboard with dataset insights
`/predict`	GET, POST	Input form (GET) and prediction result with charts (POST)
`/model`	GET	Model comparison and evaluation
`/train`	POST	Retrain from the current MySQL data
`/employees`	GET	Browse, search, filter and page through records
`/employees/add`, `/employees/<id>/delete`	POST	Add or delete a record
`/employees.csv`	GET	Download all records as CSV
`/history`, `/history/clear`	GET, POST	Prediction history and clearing it

10.1 Dashboard



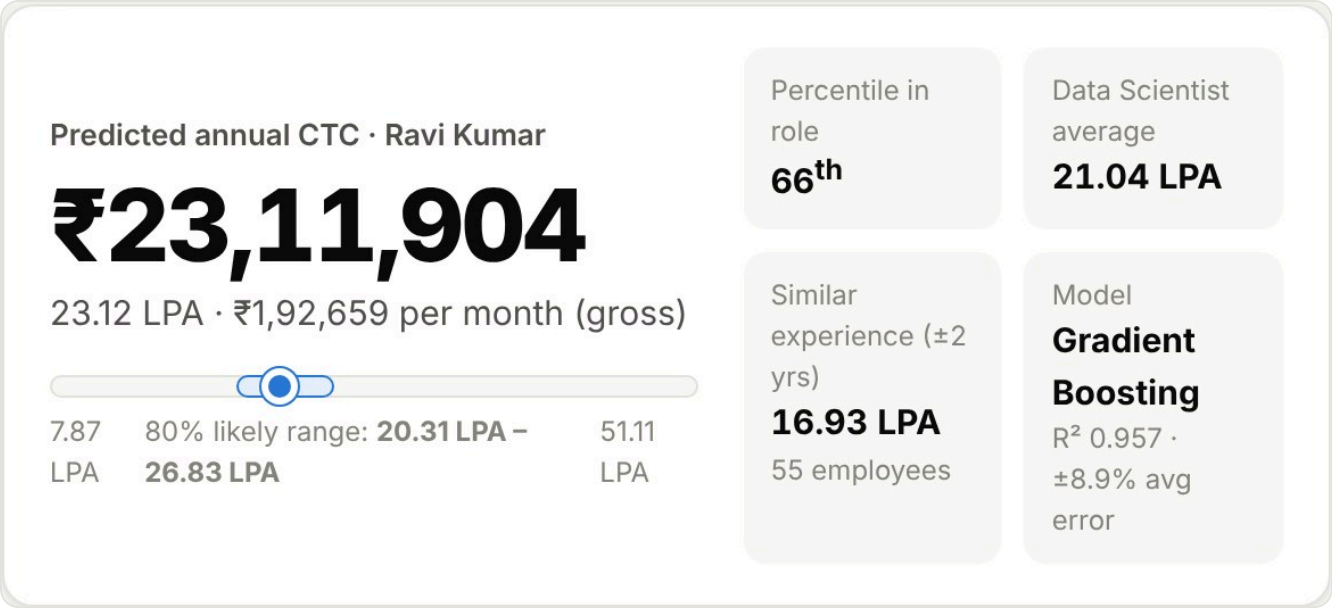
Dashboard: KPI tiles, salary vs experience scatter (filter by role), distribution, experience trend, role and education averages, skill premium and location × company comparison.

10.2 Predict salary – input screen

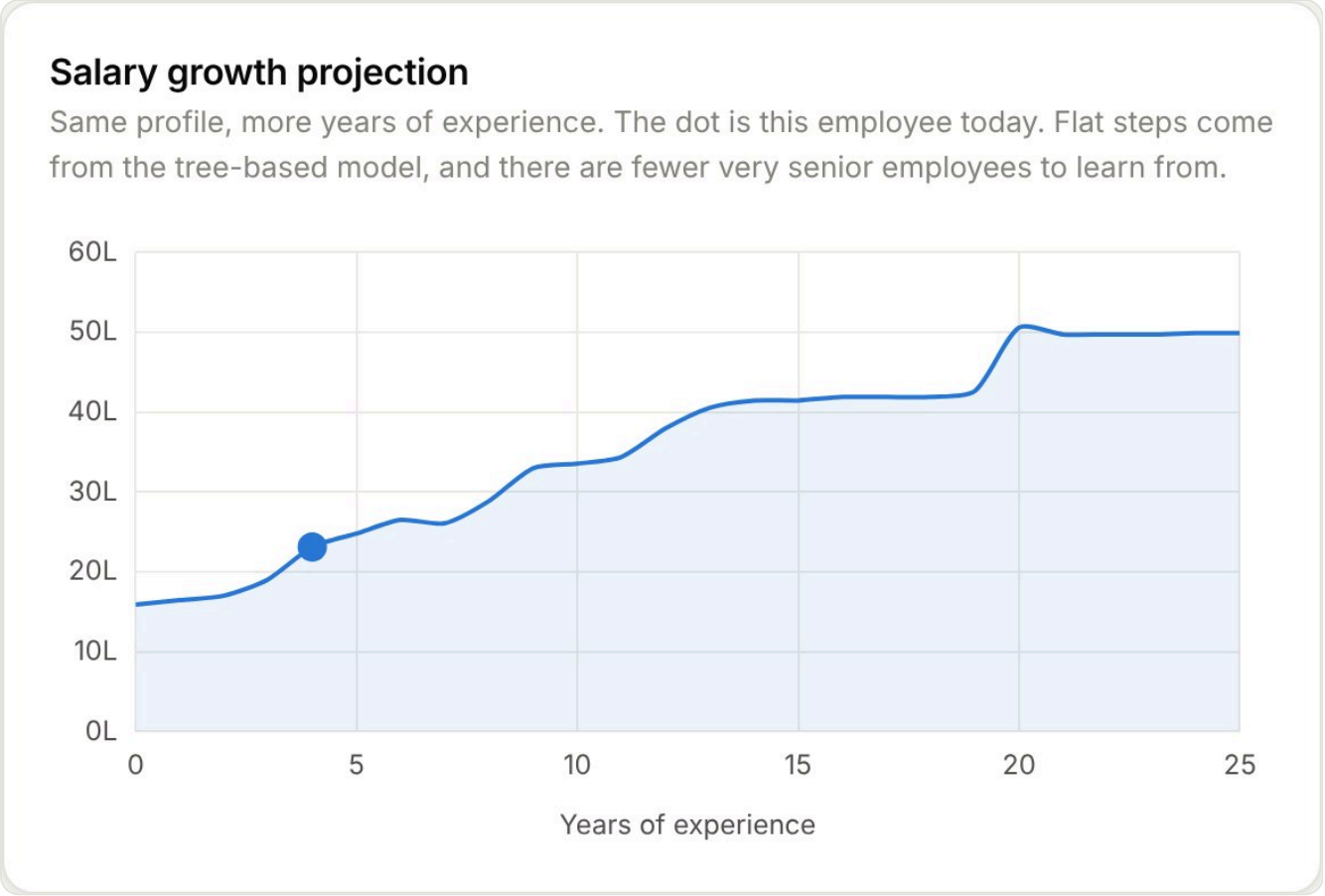


Input form: experience slider, drop-downs, certification count, star rating and skill chips.

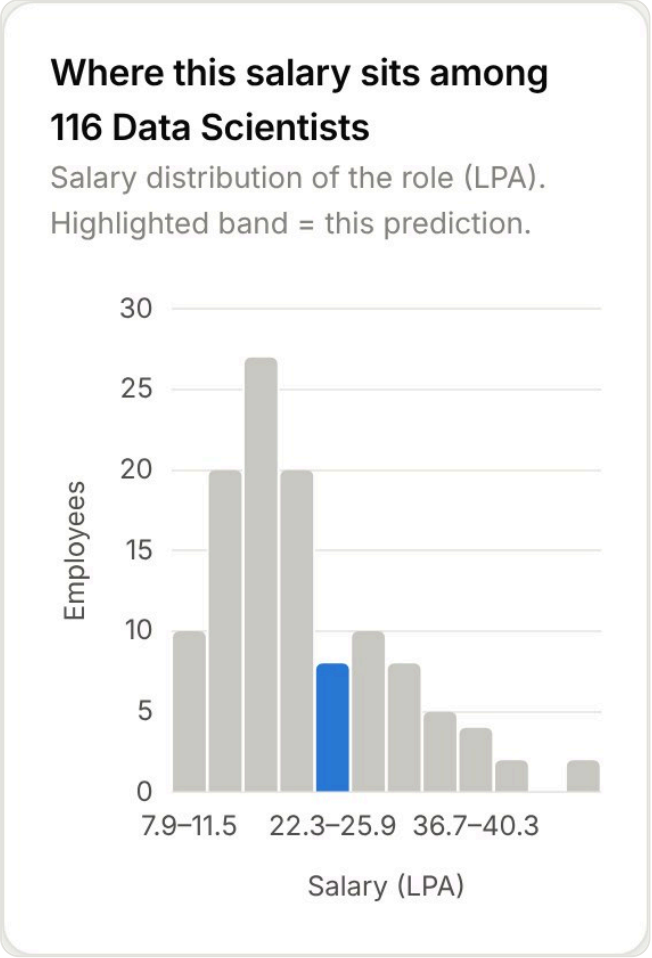
10.3 Predict salary – output screen



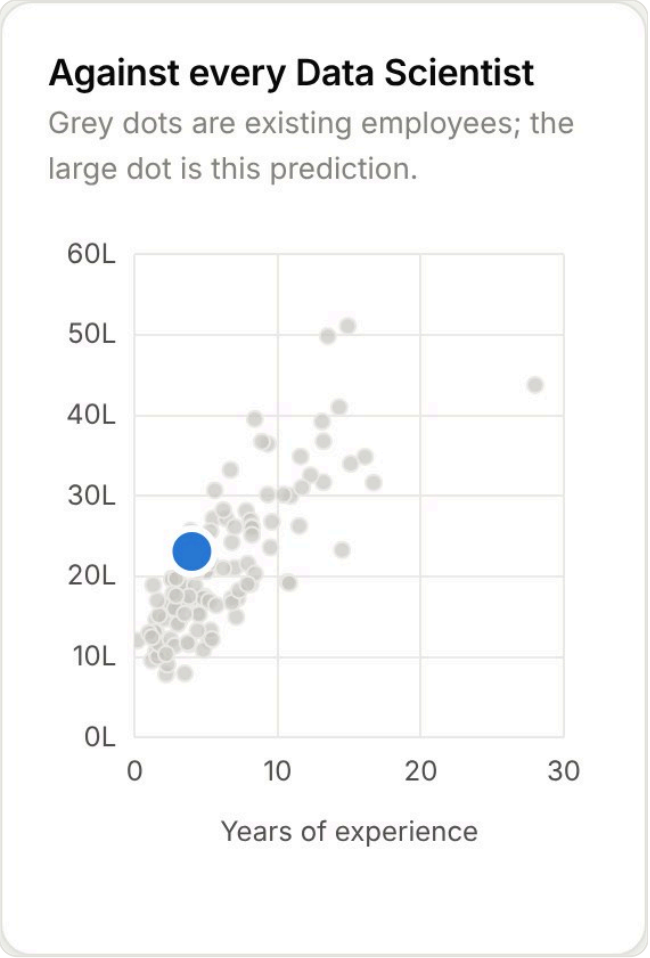
Result: predicted CTC ₹23.12 L (₹1.93 L per month), 80% range 20.3–26.8 L shown on the role's salary span, 66th percentile among Data Scientists.



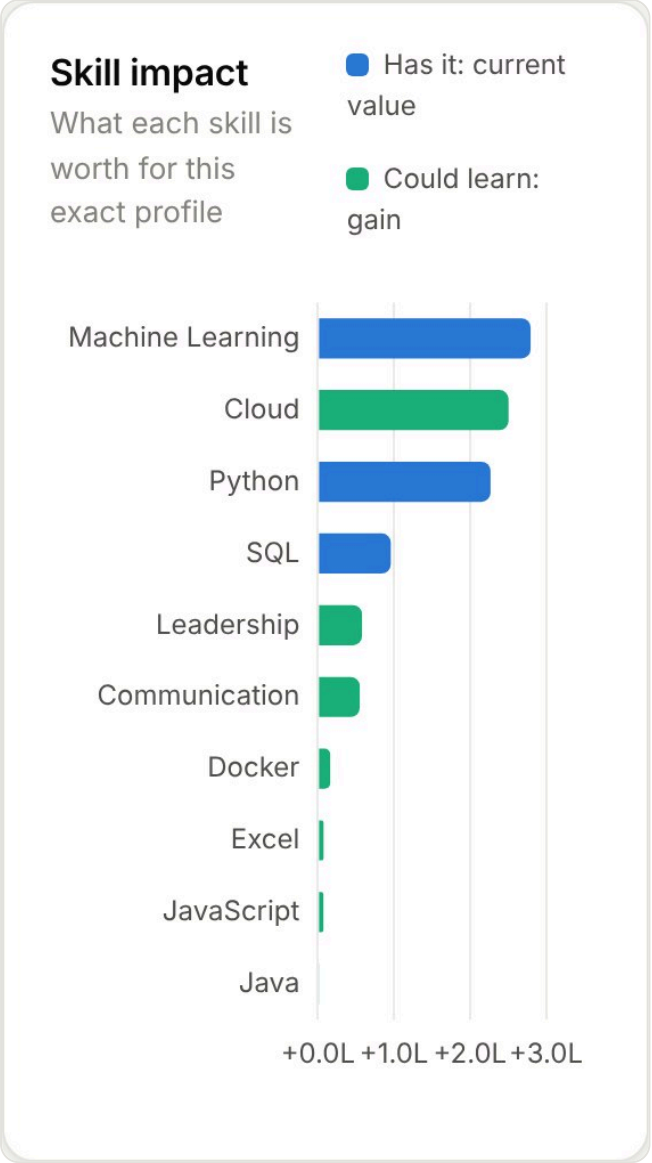
Salary growth projection: the same profile with 0–25 years of experience. The dot marks today.



Peer distribution: where this salary falls among all Data Scientists.



Peer scatter: this prediction against every Data Scientist.



Skill impact: blue shows the current value of skills held; green shows the gain from learning a new one.

How to earn more

Best skills to add next, with the predicted raise

Learn **Cloud**	**+₹2,50,771** (+10.8%)
Learn **Leadership**	**+₹58,021** (+2.5%)
Learn **Communication**	**+₹55,024** (+2.4%)

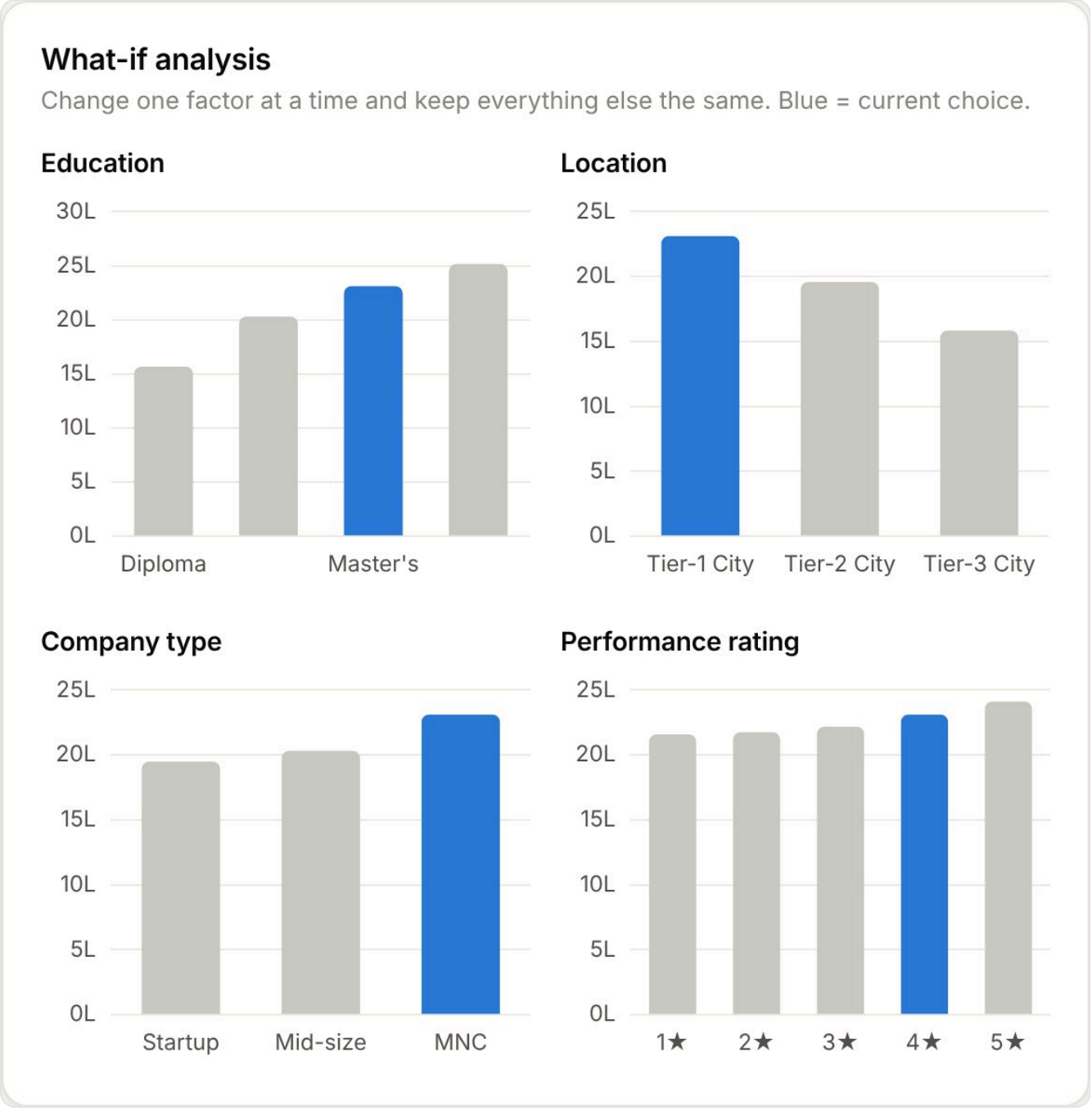
Current skills

Python · SQL · Machine Learning

Profile

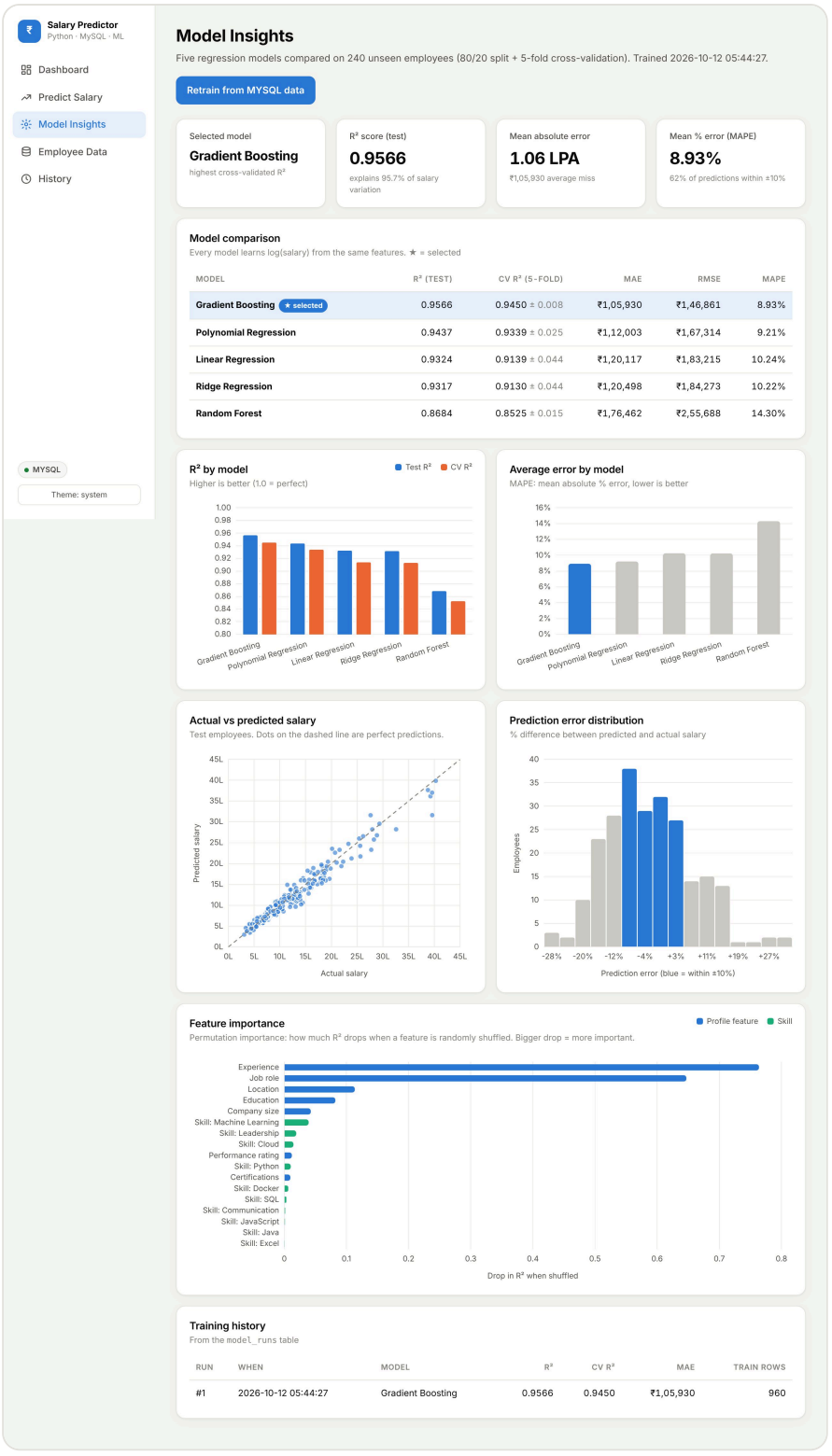
Data Scientist · 4.0 yrs · Master's · Tier-1 City · MNC · 2 certifications · rating 4/5

How to earn more: the top 3 skills to learn next, with the rupee gain.



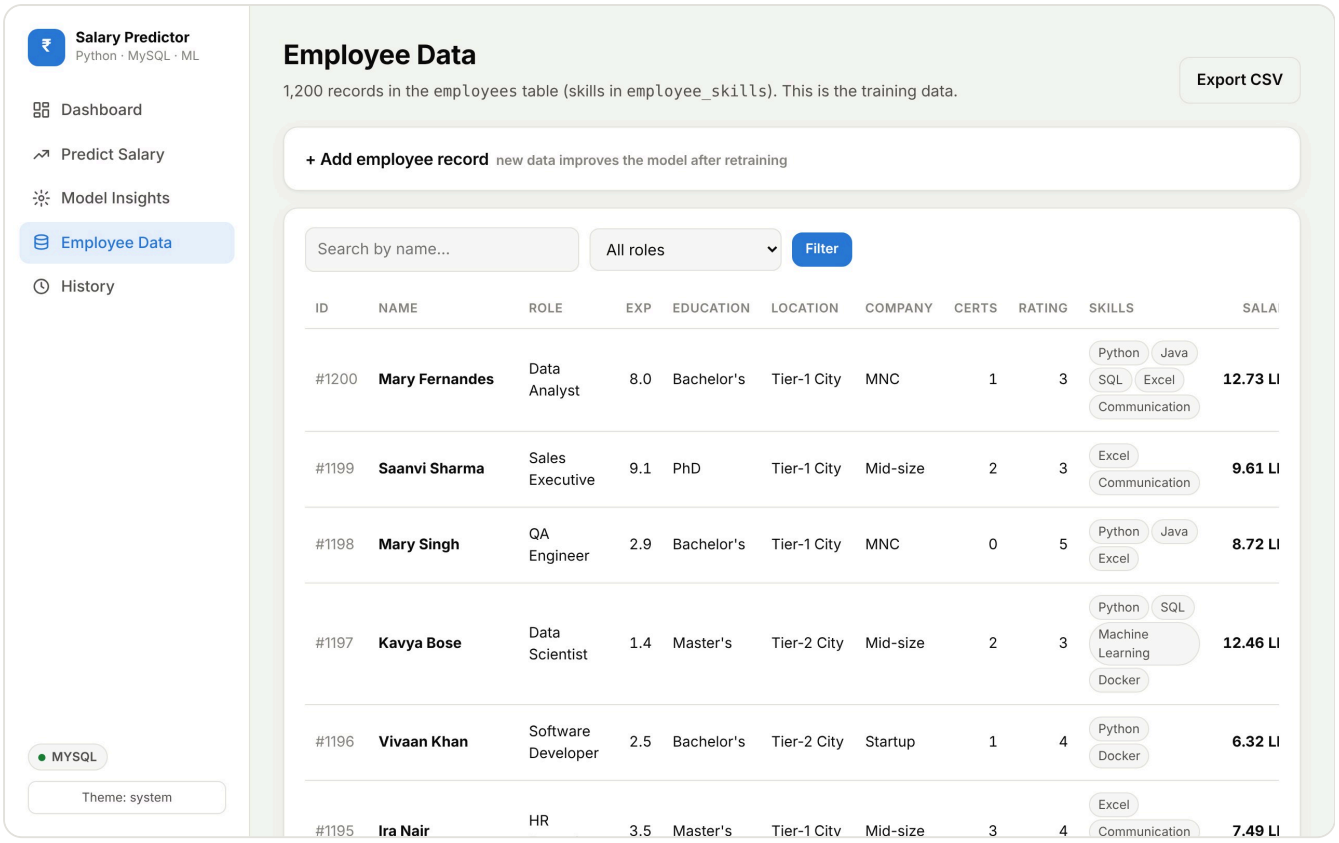
What-if analysis: change one factor at a time (education, location, company type, rating). Blue is the current choice.

10.4 Model insights

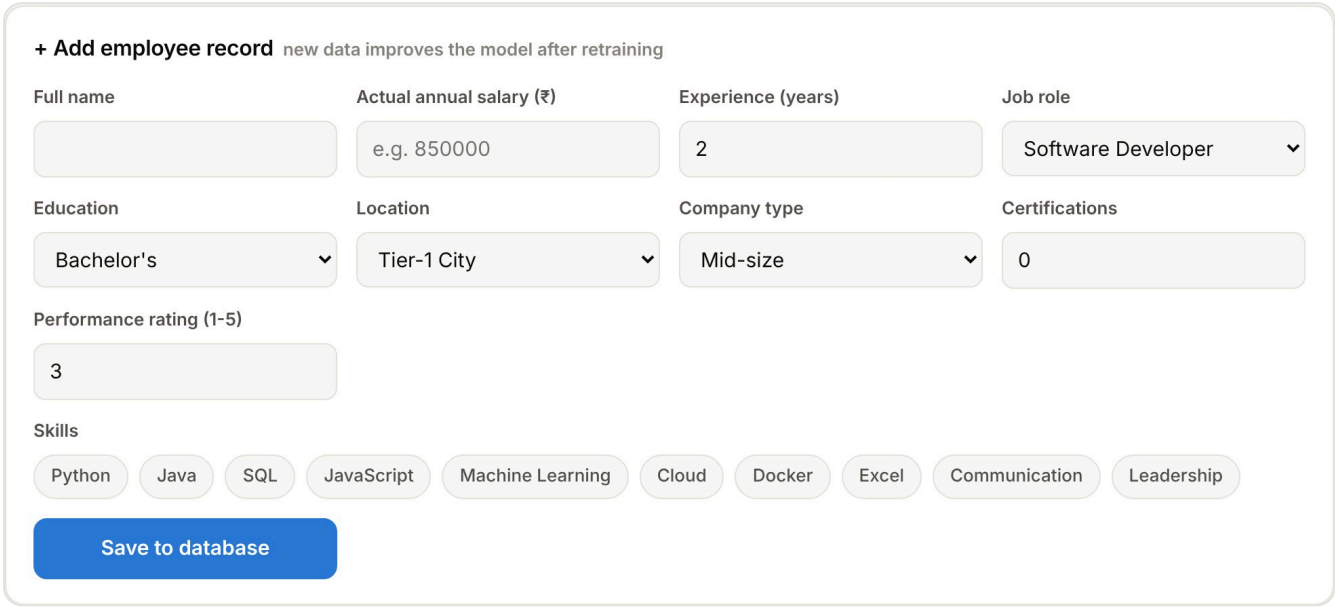


Model Insights page: KPI tiles, comparison table, R² and MAPE charts, accuracy plots, feature importance, training history and the Retrain button.

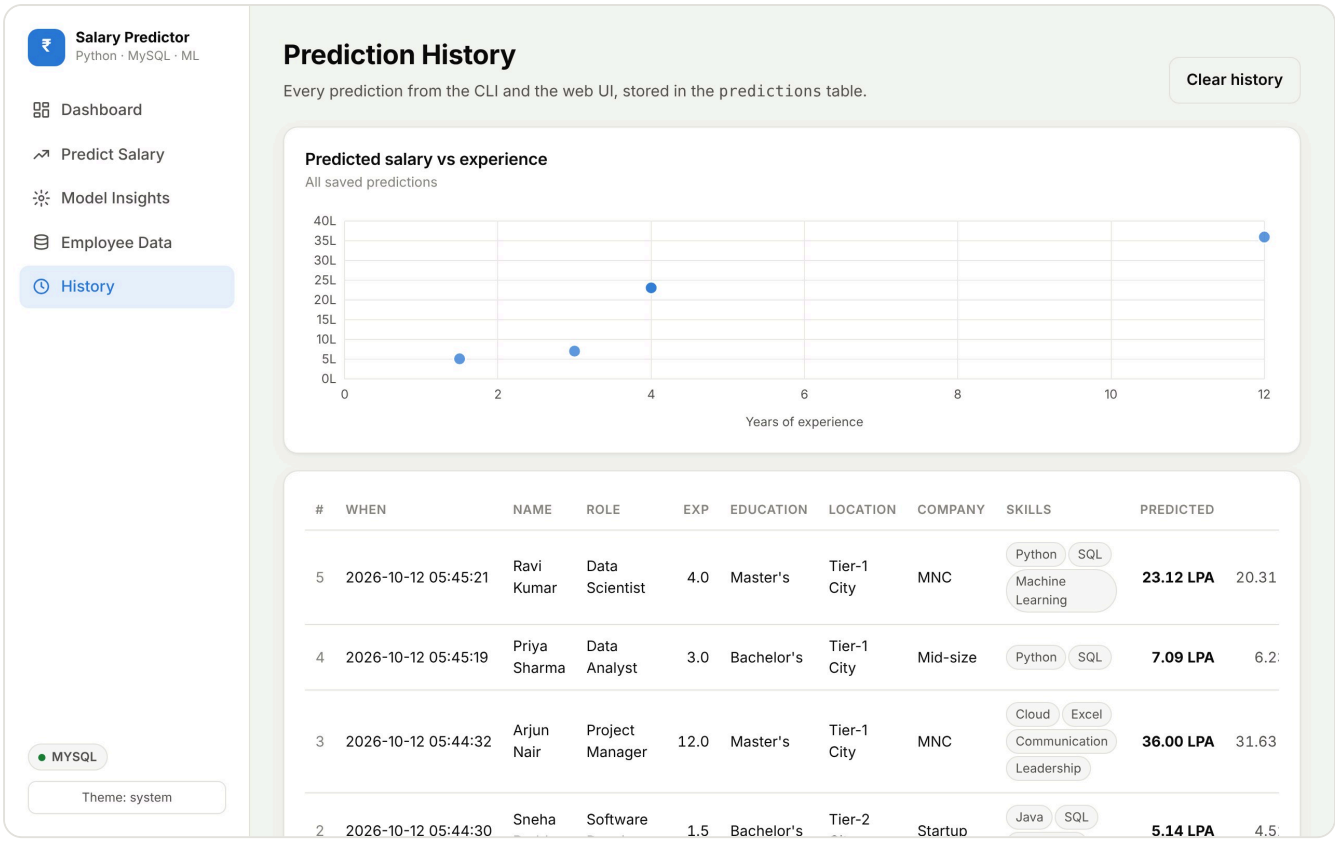
10.5 Employee data & history



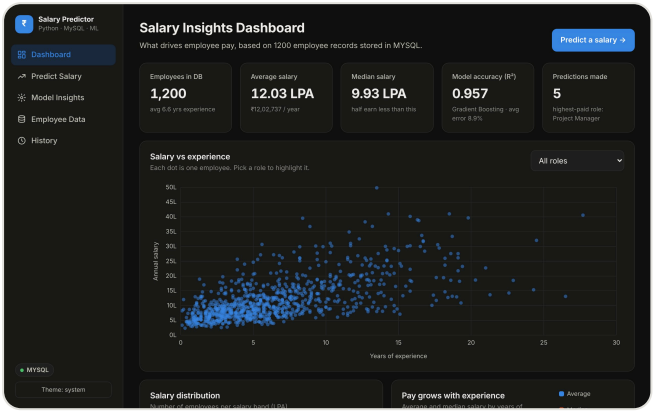
Employee Data: records from the `employees` table with search, role filter, paging, delete and CSV export.



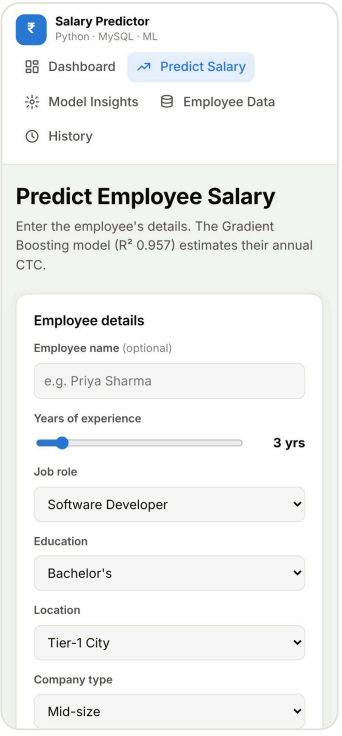
Add employee: new records are saved to MySQL and used at the next retrain.



Prediction History: predictions from both the CLI and the web, read from the `predictions` table.



Dark theme: theme toggle in the sidebar.



Mobile layout: the same app on a phone.

11 Installation & Running

11.1 Windows – one step

1. *(Optional)* Start MySQL (MySQL Server, or click **Start** next to MySQL in XAMPP).
2. Open the `salary_prediction` folder and **double-click** `START_HERE.bat`.

On the first run it installs Python 3.12 with `winget` if needed, creates a private virtual environment (`.venv`), installs the packages, asks for the MySQL login once (answer N to use SQLite), creates the tables, loads the data and trains the model. It then shows this menu:

```
  1. Web version           (opens in your browser)
  2. Command-line version
  3. Change MySQL settings
  4. Reset database and retrain model
  5. Reinstall packages
  0. Exit
```

11.2 Manual (any OS)

```
cd salary_prediction
pip install -r requirements.txt
python configure_db.py        # MySQL login (or set MYSQL_USER / MYSQL_PASSWORD)
python setup_db.py            # tables + 1,200 records + train model
python cli_app.py             # Version 1
python web_app.py             # Version 2 → http://127.0.0.1:5000
```

11.3 Project structure

```
salary_prediction/
├── START_HERE.bat       Windows one-click installer + launcher
├── config.py            settings, feature lists, ₹ formatting
├── schema.sql           MySQL schema (5 tables)
├── db.py                all SQL (MySQL with SQLite fallback)
├── data_generator.py    sample dataset generator
├── ml_model.py          training, evaluation, prediction, what-if
├── setup_db.py          create → seed → train
├── configure_db.py      MySQL login wizard
├── cli_app.py           VERSION 1 – command line
├── web_app.py           VERSION 2 – Flask web app
├── templates/           base, dashboard, predict, model, employees, history
├── static/              style.css, charts.js, chart.umd.min.js
└── requirements.txt
```

12 Testing

#	Test case	Input	Expected / actual result
1	Valid prediction (CLI)	4 yrs, Master's, Data Scientist, MNC, Python+SQL+ML	₹23.12 L shown with range and charts, saved to DB ✔
2	Valid prediction (Web)	Same as above via form	Same value as CLI, all 8 chart cards render ✔
3	Non-numeric experience	`experience = abc`	"Experience Years must be a number" ✔
4	Out-of-range value	rating = 9, experience = 50	Range error message, no crash ✔
5	Invalid option	role = "Pilot"	"Job Role must be one of ..." ✔
6	No skills selected	empty skills	Prediction works, all skills shown as "could learn" ✔
7	Add employee	Name, details, salary ₹7.2 L	Row in `employees` + `employee_skills` ✔
8	Add employee with bad salary	salary = 100	"Salary must be between ..." ✔
9	Retrain	Click Retrain / menu 4	New row in `model_runs`, metrics refreshed ✔
10	Search & filter	name "ra" + role filter	Only matching rows, paging works ✔
11	CSV export	/employees.csv	1,200 rows downloaded ✔
12	SQL injection attempt	search `' OR 1=1 --`	Treated as text, no extra rows ✔
13	MySQL wrong password	configure_db with wrong password	"Access denied", offered retry or SQLite ✔
14	MySQL not running	server stopped	Falls back to SQLite automatically ✔
15	All web routes	GET every page	HTTP 200, no browser console errors ✔
16	Mobile and dark mode	390 px width, dark theme	No horizontal scroll, readable charts ✔

13 Advantages, Limitations & Future Scope

13.1 Advantages

- High accuracy (R^2 0.957), with the algorithm chosen **automatically** by cross-validation.
- **Explainable**: range, percentile, feature importance and what-if, not just a number.
- Two interfaces sharing one core: CLI for quick use, Web for rich visual analysis.
- Normalised MySQL design with an audit trail of every prediction and training run.
- Improves over time: add records and retrain.
- Easy to run: a one-click Windows installer, SQLite fallback and offline charts.

13.2 Limitations

- The sample data is **synthetic**. Accuracy on real company data will differ, and the model must be retrained on it.
- It only knows the 8 roles, 10 skills and categories it was trained on.
- It cannot capture unrecorded factors (negotiation, interview performance, specific company, market changes).
- Predictions for very senior profiles (20+ years) are less reliable because the data has few of them. This shows up as flat steps in the growth curve.
- Salaries are a snapshot in time; inflation and yearly market changes need fresh data.

13.3 Future scope

- Train on real data (company HR systems, public salary surveys) and add more roles and skills.
- Add XGBoost/LightGBM, hyperparameter tuning (GridSearchCV) and SHAP explanations.
- User login with admin and employee roles, and a REST API for other systems.
- Deploy online (cloud or shared hosting with a PHP front end that reads an exported model).
- Resume upload that extracts skills automatically with NLP.
- A gender and fairness audit to detect pay bias.

14 Conclusion

The Employee Salary Prediction System shows a complete, working machine-learning application, from a normalised MySQL database through a validated ML pipeline to two user interfaces.

By comparing five regression algorithms with cross-validation, the system selected Gradient Boosting, which explains about 96% of the variation in salary with an average error under 9%. Experience and job role were confirmed as the strongest drivers of pay, followed by location, education and company type. Among skills, Machine Learning and Cloud give the highest premium.

Beyond a single number, the system explains each prediction with a likely range, a peer percentile, a growth projection and what-if analysis. That makes it useful for HR salary planning and for individuals planning their careers. Every prediction and training run is stored in MySQL, and the model can be retrained as new data arrives, so the system can keep improving in real use.

15 Viva Questions & Answers

Short, exam-ready answers. Read the bold first line of each answer if you are short of time.

A. Project overview

Q1. What is your project?

An Employee Salary Prediction System that predicts an employee's annual salary using machine learning (regression) with Python and MySQL. It takes experience, education, skills, job role, location, company type, certifications and performance rating as input and outputs the expected salary with a likely range and explanations.

Q2. Why did you choose this project?

Fixing salaries is a real, everyday problem for HR teams and job seekers. It is a classic supervised regression problem, combines a database with ML, and its results are easy to understand and check.

Q3. What is the main objective?

To predict salary accurately and explain the prediction. Other goals: store data in a normalised MySQL database, compare several algorithms, provide CLI and web interfaces, and allow retraining with new data.

Q4. Who are the end users?

HR departments (offer planning, fair pay bands), **job seekers and students** (checking their market value and which skills to learn), and **managers** (appraisal planning).

Q5. Why is this a regression problem and not classification?

Because the output, salary, is a continuous number. Classification predicts a category (e.g. Spam / Not Spam). If we predicted salary bands like Low/Medium/High it would become classification.

Q6. What type of machine learning is used?

Supervised learning. Each training example has the correct answer (the actual salary), and the model learns the mapping from features to salary.

Q7. What are the input and output screens?

Input: employee details (form in the web version, prompts in the CLI). **Output:** predicted annual and monthly salary, 80% range, percentile, growth projection, skill impact and what-if charts.

Q8. Why did you build two versions?

The CLI version shows the core logic simply and runs anywhere; the web version gives a user-friendly interface with rich visualisation. Both share the same database and ML code, so there is no duplicated logic.

Q9. What technologies did you use?

Python, MySQL, pandas, NumPy, scikit-learn, joblib, Flask, Jinja2, HTML/CSS, Chart.js and mysql-connector-python.

Q10. What is the overall architecture?

A layered architecture: presentation layer (CLI and web), business/ML layer (`ml_model.py`), data access layer (`db.py`) and the MySQL database. Each layer talks only to the layer below it.

Q11. What is unique in your project compared to a basic salary predictor?

It compares 5 algorithms automatically, gives a prediction range instead of only one number, and explains the result with percentile, feature importance and what-if analysis (skill, education, city and company impact). It also logs every prediction and training run in MySQL and has a one-click installer.

Q12. What is the accuracy of your model?

R^2 = 0.957 on unseen test data, with a mean absolute percentage error of 8.9% and MAE of about ₹1.06 lakh. The 5-fold cross-validated R^2 is 0.945.

B. Dataset & features

Q13. Where did you get the dataset?

Real salary data is confidential, so we generated a realistic synthetic dataset of 1,200 employees modelled on the Indian IT and corporate market. The salary depends on role, experience, education, skills, city, company type, certifications and rating, plus random noise. The ML model never sees this formula; it learns the pattern only from the rows in MySQL.

Q14. Is synthetic data acceptable? Will it work on real data?

Yes, for demonstrating the method. The pipeline does not depend on the data source. Inserting real records into the `employees` and `employee_skills` tables and retraining is enough. Accuracy on real data would differ and must be measured again.

Q15. Why did you add noise to the data?

Real salaries are never a perfect formula. The noise (about ±9%, log-normal) stands in for factors we don't record, such as negotiation or interview performance. Without noise any model would score almost 100%, which is unrealistic.

Q16. What are your features?

14 inputs: experience, certifications, performance rating (numeric); education, job role, location, company size (categorical); and 10 skills as 0/1 flags. After encoding they become 31 numeric columns.

Q17. What is the target variable?

`salary` , the annual CTC in rupees, stored in the `employees` table. The model internally learns log(salary).

Q18. What is the difference between a feature and a label?

Features are the inputs (X); the label/target is what we predict (y). Here X = employee details and y = salary.

Q19. What is the distribution of salary in your data?

Right-skewed: many employees between 5 and 12 LPA and a long tail up to 51 LPA. Mean 12.03 LPA, median 9.93 LPA. The mean is above the median because of the high earners.

Q20. How did you handle missing values?

The generated data has no missing values, and input validation prevents missing inputs. An employee with no skills is handled by the LEFT JOIN (NULL becomes an empty skill list). Unknown categories are ignored by `OneHotEncoder(handle_unknown='ignore')`.

Q21. How are skills represented for the model?

Multi-hot encoding: one column per skill (`skill_python`, `skill_cloud`, ...) with 1 if the employee has it and 0 otherwise, because one employee can have several skills.

Q22. What is data preprocessing and why is it needed?

Converting raw data into a clean numeric form that algorithms can use: encoding categories, scaling numbers, transforming the target and validating ranges. Models only understand numbers, and good preprocessing improves accuracy.

C. Database & MySQL

Q23. Why MySQL?

MySQL is a free, reliable and widely used relational database. It stores structured data in tables, enforces relationships with foreign keys, supports SQL queries and works easily with Python.

Q24. How many tables are there? Name them.

Five: `employees`, `skills`, `employee_skills`, `predictions` and `model_runs`.

Q25. Why is there a separate skills table?

Because employees and skills have a many-to-many relationship. Putting a comma-separated list in one column would break First Normal Form and make queries and updates hard. A junction table (`employee_skills`) is the correct design.

Q26. What is normalisation? Which normal form is your database in?

Organising tables to reduce redundancy and avoid insert/update/delete anomalies. Our design is in **3NF**: atomic values (1NF), no partial dependency on a composite key (2NF) and no transitive dependency (3NF).

Q27. What is a primary key? Give an example.

A column (or set of columns) that uniquely identifies each row and cannot be NULL. Example: `employee_id` in `employees`, which is AUTO_INCREMENT.

Q28. What is a foreign key?

A column that refers to the primary key of another table, enforcing referential integrity. `employee_skills.employee_id` refers to `employees.employee_id`.

Q29. What is a composite key?

A primary key made of two or more columns. `employee_skills` uses (employee_id, skill_id), so the same skill cannot be linked to the same employee twice.

Q30. What does ON DELETE CASCADE do?

When a parent row is deleted, the matching child rows are deleted automatically. Deleting an employee removes their rows from `employee_skills`.

Q31. What is AUTO_INCREMENT?

MySQL automatically generates the next integer ID for each new row, so we never have to supply `employee_id` ourselves.

Q32. Why DECIMAL for salary instead of FLOAT?

DECIMAL stores exact values, which matters for money. FLOAT is approximate and can introduce rounding errors.

Q33. Explain the query used to load training data.

It LEFT JOINs employees → employee_skills → skills and uses GROUP_CONCAT to collect each employee's skills into one string, grouped by employee. pandas then splits that string into 10 skill columns.

Q34. Difference between INNER JOIN and LEFT JOIN?

INNER JOIN returns only rows that match in both tables; LEFT JOIN returns all rows from the left table, with NULLs where there is no match. We use LEFT JOIN so employees with zero skills are not lost.

Q35. What is GROUP_CONCAT?

An aggregate function that joins values from several rows into one comma-separated string, e.g. 'Python,SQL,Machine Learning'.

Q36. How do you prevent SQL injection?

With parameterised queries: placeholders (`%s`) with values passed separately to `cursor.execute(sql, params)`. The driver escapes them, so user input is always treated as data, never as SQL code.

Q37. How does Python connect to MySQL?

Using the `mysql-connector-python` driver: `mysql.connector.connect(host, port, user, password, database)` returns a connection; we create a cursor, execute SQL, fetch results and commit.

Q38. What happens if MySQL is not running?

The app falls back to SQLite automatically (the same tables in a local file) and shows which backend is in use, so the demo still works. The MySQL login is stored in `db_settings.ini`.

Q39. What is the purpose of the predictions and model_runs tables?

Auditing and tracking. `predictions` keeps every prediction (inputs, output, range, source); `model_runs` keeps the metrics of every training run so we can see whether the model got better or worse.

Q40. What is the difference between DDL and DML? Give examples from your project.

DDL defines structure; DML changes data. DDL: `CREATE DATABASE`, `CREATE TABLE`, `DROP TABLE`. DML: `INSERT` (add employee or prediction), `SELECT`, `DELETE`.

D. Python & libraries

Q41. What is pandas used for?

Handling tabular data as DataFrames: loading query results, creating skill columns, grouping (average salary by role or education) and sampling for charts.

Q42. What is NumPy used for?

Fast numerical operations: log/exp transforms, histograms, percentiles (quantiles for the prediction range) and random number generation for the dataset.

Q43. What is scikit-learn?

Python's standard machine-learning library. We use it for preprocessing (OneHotEncoder, StandardScaler, PolynomialFeatures), models, Pipelines, train_test_split, cross_val_score, metrics and permutation_importance.

Q44. What is joblib and why use it?

A library for saving and loading Python objects efficiently, especially large NumPy arrays. We save the trained pipeline to `salary_model.joblib` so we don't retrain on every prediction.

Q45. What is a scikit-learn Pipeline?

A chain of preprocessing steps and a model that behaves like a single estimator. It guarantees the same transformations at training and prediction time and prevents data leakage during cross-validation.

Q46. What is ColumnTransformer?

It applies different preprocessing to different columns: StandardScaler for numeric columns, OneHotEncoder for categorical columns and passthrough for the skill flags.

Q47. What is TransformedTargetRegressor?

A wrapper that transforms the target before training and inverts it after prediction. We use np.log and np.exp, so the model learns log(salary) but returns rupees.

Q48. What is argparse used for?

Parsing command-line arguments, so the CLI can run commands like `python cli_app.py predict --experience 4 --role "Data Scientist"` without the menu.

Q49. What is a virtual environment (.venv)?

An isolated Python installation for one project, so its package versions don't conflict with other projects. START_HERE.bat creates one automatically.

Q50. What does random_state=42 do?

It fixes the random seed so splits and models give the same results every run, which makes results reproducible.

E. Machine learning fundamentals

Q51. What is machine learning?

A field of AI where computers learn patterns from data instead of following hand-written rules, then use them to make predictions on new data.

Q52. Explain the ML workflow you followed.

Collect data (MySQL) → preprocess → split train/test → train several models → cross-validate → select the best → evaluate on the test set → save → predict → monitor and retrain.

Q53. What is a train-test split? What ratio did you use?

The data is divided so the model is evaluated on examples it never saw. We used 80/20: 960 training rows and 240 test rows.

Q54. What is cross-validation?

A more reliable evaluation: the training data is split into k parts (k = 5); the model trains on k−1 and validates on the remaining part, repeated k times. The average score is used to compare models, so the result doesn't depend on one lucky split.

Q55. Why choose the model by CV score and not test score?

Choosing by test score would leak test information into the decision and make the reported test score optimistic. The test set should only be used for the final, unbiased report.

Q56. What is overfitting? How did you avoid it?

Overfitting is when a model memorises the training data, noise included, and performs poorly on new data. We avoided it with cross-validation, a held-out test set, Ridge regularisation, shallow trees (depth 3), a small learning rate, subsampling and min_samples_leaf in Random Forest.

Q57. What is underfitting?

When a model is too simple to capture the pattern, so it performs poorly even on training data. For example, plain linear regression can't capture the curved growth of salary with experience.

Q58. What is the bias-variance trade-off?

Bias is error from overly simple assumptions (underfitting); variance is error from sensitivity to the training data (overfitting). A good model balances both. Boosting reduces bias step by step, and the learning rate and shallow trees control variance.

Q59. What is one-hot encoding? Why not label encoding?

One-hot creates a 0/1 column for each category. Label encoding (Diploma = 0, Bachelor's = 1, ...) would make models assume numeric distances and an order, e.g. that 'Sales' is twice 'HR', which is wrong for nominal categories like role or city.

Q60. What is feature scaling? Which scaler did you use?

Putting numeric features on a similar scale. We used StandardScaler: z = (x − mean) / std. It matters for linear models with regularisation; tree models don't need it.

Q61. Why did you apply a log transform to salary?

Salary effects are multiplicative (percentages) and the distribution is right-skewed. In log space the effects become additive, the distribution becomes more symmetric, and errors become relative. That improved the linear models a lot and keeps predictions positive.

Q62. What is data leakage?

When information from the test data (or the future) gets into training, giving overly optimistic results. Pipelines fit the scaler and encoder only on training folds, which prevents it.

Q63. What is a hyperparameter? Give examples.

A setting chosen before training, not learned from data. Examples: n_estimators = 400, learning_rate = 0.05 and max_depth = 3 for Gradient Boosting; alpha = 1.0 for Ridge.

Q64. What is regularisation?

Adding a penalty for model complexity to reduce overfitting. Ridge adds $\alpha \cdot \Sigma b^2$ (L2); Lasso adds $\alpha \cdot \Sigma |b|$ (L1) and can shrink some coefficients to zero.

Q65. What is a residual?

Residual = actual − predicted. Gradient Boosting builds each new tree to predict the residuals of the previous trees.

Q66. What is feature engineering in your project?

Creating useful inputs: skill flags from the skills table, polynomial experience terms (experience², experience³), role × experience interactions in Polynomial Regression, and the log of the target.

F. Algorithms

Q67. Explain Linear Regression.

It fits a straight-line (hyperplane) relationship $\hat{y} = b_o + b_1x_1 + \ldots + b_nx_n$ by minimising the sum of squared errors (Ordinary Least Squares). Each coefficient shows how much y changes when that feature rises by one unit.

Q68. What is Ridge Regression? How is it different from Linear?

Linear regression with an L2 penalty ($\alpha \cdot \Sigma b^2$) that shrinks coefficients. It reduces overfitting and handles correlated features better. With α = 0 it becomes plain linear regression.

Q69. What is Polynomial Regression?

Linear regression on expanded features such as x^2, x^3 and products of features. It can fit curves (salary growth flattening) and interactions (some roles grow faster). It is still linear in its coefficients.

Q70. What is a Decision Tree?

A model that splits data with if-else questions (e.g. 'experience < 5?') into leaves. For regression, each leaf predicts the average target of its training samples. Splits are chosen to reduce the squared error the most.

Q71. Explain Random Forest.

An ensemble of many decision trees, each trained on a bootstrap sample with random feature subsets; the prediction is their average (bagging). Averaging reduces variance and overfitting.

Q72. Explain Gradient Boosting.

An ensemble that builds trees one after another; each new small tree learns to correct the residual errors of the trees so far. Prediction = initial guess + learning_rate × (sum of all trees' outputs). We used 400 trees, depth 3, learning rate 0.05 and subsample 0.9.

Q73. Difference between bagging and boosting?

Bagging (Random Forest) trains trees independently in parallel and averages them, which reduces variance. Boosting (Gradient Boosting) trains trees sequentially, each fixing the previous errors, which reduces bias.

Q74. Why is it called 'gradient' boosting?

Each new tree is fitted to the negative gradient of the loss function. For squared error loss that gradient is simply the residual, so boosting performs gradient descent in function space.

Q75. What is the learning rate in boosting?

A factor (0.05 here) that scales each tree's contribution. Smaller rates need more trees but generalise better.

Q76. Why did Gradient Boosting perform best?

It captures non-linear effects (curved experience growth) and interactions (role × experience) automatically, while its sequential error correction keeps bias low. Shallow trees, a small learning rate and subsampling stop it from overfitting. It is usually the strongest algorithm on tabular data.

Q77. Why did Random Forest perform worst here?

Averaging many trees pulls predictions toward the middle, so it under-predicts the rare very high salaries (the right tail). It also doesn't benefit as much from the log-scale additive structure that boosting and linear models exploit.

Q78. Why do linear models score so high (0.93)?

Because of the log transform: multiplicative effects like ×1.18 for a Master's become additive in log space, which suits a linear model.

Q79. What are other algorithms you could use?

XGBoost, LightGBM, CatBoost, Support Vector Regression, K-Nearest Neighbours, Lasso/ElasticNet and neural networks (MLPRegressor).

Q80. Why not deep learning?

With about 1,000 rows of tabular data, tree ensembles are more accurate, faster and easier to explain. Neural networks need much more data and tuning.

G. Evaluation metrics & validation

Q81. What is R^2 score?

The proportion of the variance in the target explained by the model: $R^2 = 1 - SS_{res} / SS_{tot}$. 1 is perfect; 0 equals always predicting the mean; it can be negative for very bad models. Ours is 0.957.

Q82. What is MAE?

Mean Absolute Error, the average of |actual – predicted|, in rupees. Ours is about ₹1,05,930. It is easy to explain to non-technical people.

Q83. What is RMSE? How is it different from MAE?

Root Mean Squared Error, the square root of the average squared error. Squaring penalises large errors more, so RMSE ≥ MAE. Ours is ₹1,46,861.

Q84. What is MAPE?

Mean Absolute Percentage Error, the average of |error| / actual × 100. Ours is 8.93%. It is fair across low and high salaries: a ₹50k miss matters more on a ₹3L salary than on a ₹30L one.

Q85. Why use several metrics instead of one?

Each shows a different aspect: R^2 = how much is explained, MAE = typical miss in ₹, RMSE = sensitivity to big misses, MAPE = relative miss. Together they give a complete picture.

Q86. Why not use accuracy?

Accuracy (percentage of exactly correct predictions) is a classification metric. A continuous salary is almost never exactly right, so regression uses error-based metrics.

Q87. What does CV R^2 0.945 ± 0.008 mean?

The average R^2 across the 5 folds was 0.945, with a standard deviation of 0.008. The small spread means the model is stable whichever data it trains on.

Q88. How is the 80% prediction range calculated?

From the test set, we take the ratio actual/predicted for every employee and find its 10th and 90th percentiles (0.879 and 1.160). For a new prediction P, the range is 0.879P to 1.160P, so about 80% of real salaries fall inside it.

Q89. What is permutation feature importance?

Shuffle one feature's values in the test set and measure how much R^2 drops. A big drop means the model depends on that feature. It works for any model and is measured on unseen data.

Q90. Which features are most important?

Experience (0.76) and job role (0.65), then location (0.11), education (0.08) and company size (0.04). Machine Learning (0.04) is the most important single skill.

Q91. Why is the actual vs predicted plot useful?

Perfect predictions lie on the diagonal y = x. Points close to the line show good accuracy, and systematic deviations, such as under-predicting high salaries, show bias.

Q92. What does the error distribution chart show?

A histogram of percentage errors centred near 0%, roughly symmetric. This means no systematic over- or under-prediction; about 62% of predictions are within ±10%.

H. Implementation – CLI & Web

Q93. What is Flask?

A lightweight Python web framework. We define routes (URLs) with decorators like `@app.route('/predict', methods=['GET','POST'])`, process requests and return HTML rendered from templates.

Q94. What is Jinja2?

Flask's template engine. It lets HTML contain variables, loops and conditions (`{{ result.predicted|inr }}` , `{% for r in rows %}`), and supports template inheritance through `base.html` .

Q95. Difference between GET and POST?

GET requests data (it can be bookmarked; parameters go in the URL); POST sends form data in the request body to change something. /predict uses GET to show the form and POST to submit it; add, delete and retrain use POST.

Q96. How are charts drawn in the web version?

Flask passes the data to the page as JSON (the `tojson` filter), and Chart.js (JavaScript) draws interactive canvas charts with tooltips. Chart.js is bundled locally, so it works offline.

Q97. How are charts drawn in the CLI?

As text bar charts using the '█' character, with bar length proportional to the value, plus histograms from `numpy.histogram` . The user's own value is marked '◀ you'.

Q98. How do you validate user input?

A shared `validate_input()` function checks numeric ranges (experience 0–40, rating 1–5), allowed options and known skills. It raises ValueError with clear messages; the CLI asks again and the web page shows a red message.

Q99. How is the model loaded in the web app?

It is loaded once from the joblib file and cached in memory (`_model_cache`), so each request predicts instantly. Retraining replaces the cached model.

Q100. How does the growth projection work?

The same profile is predicted 26 times with experience set to 0, 1, 2, ... 25 years, and the results are plotted as a line, with a dot at the current experience.

Q101. How does skill impact work?

For each skill, predict with it added (if missing) or removed (if present) and compare with the base prediction. The difference is the rupee value of that skill for this exact profile.

Q102. How is the percentile computed?

The percentage of employees in the database with the same job role who earn less than the predicted salary.

Q103. What does START_HERE.bat do?

A Windows batch script that installs everything in one step: it finds or installs Python (winget), creates .venv, installs packages, configures MySQL, seeds data, trains the model and then shows a menu to run the web or CLI version.

I. Limitations, deployment & tricky questions

Q104. What are the limitations of your project?

Synthetic data, a fixed set of roles and skills, no unrecorded factors (negotiation, company brand), fewer senior samples, and salaries that change over time. Retraining on real, recent data fixes most of these.

Q105. Why does the growth curve have flat steps?

Gradient Boosting is made of decision trees, which produce step-wise outputs, and there are few employees with 20+ years to learn from. A polynomial model would give a smoother curve but was less accurate overall.

Q106. Can the model predict for a new job role, e.g. 'Pilot'?

No. Validation rejects unknown roles; and even if one reached the model, OneHotEncoder would ignore it. We would need training data for that role and a retrain.

Q107. Can the model be biased?

Yes. A model learns whatever patterns exist in its data, including unfair ones. We don't use gender, religion or similar attributes, but with real data a fairness audit is still recommended.

Q108. How would you improve accuracy?

More and real data, more features (company name, domain, notice period, last salary), hyperparameter tuning (GridSearchCV), and XGBoost/LightGBM.

Q109. How can you deploy this project online?

On a cloud platform or VPS running Python (Render, Railway, PythonAnywhere, AWS) with a hosted MySQL database. On PHP-only shared hosting, export the trained model to JSON and run predictions in PHP.

Q110. What happens when a new employee is added?

The record is stored in MySQL immediately but the model doesn't change until it is retrained (menu option 4 or the Retrain button). Retraining logs a new row in model_runs.

Q111. If two employees have identical details, will they get the same prediction?

Yes, the model is deterministic. Their real salaries may differ because of unrecorded factors, which is exactly why we show a range.

Q112. Is a high R^2 always good?

Not always. A very high training R^2 with a low test R^2 means overfitting. That's why we report R^2 on unseen test data and cross-validation, along with error metrics in rupees and percent.

Q113. What did you learn from this project?

The full ML lifecycle: database design and SQL, data preprocessing, comparing and validating models, avoiding overfitting and leakage, explaining predictions, and building both CLI and web interfaces on one shared core.

Q114. Explain your project in one minute.

See the 30-second explanation in the Quick Revision Sheet. Mention the problem, the MySQL design, the 5 models with CV, Gradient Boosting with R^2 0.957, the two versions, and the explanation features (range, percentile, what-if).

16 Quick Revision Sheet

Topic	Remember this
Project	Employee Salary Prediction System, a regression problem, Python + MySQL
Input	Experience, education, job role, location, company size, certifications, rating, 10 skills
Output	Annual salary (₹) + monthly + 80% range + percentile + charts
Dataset	1,200 synthetic employees; average 12.03 LPA; median 9.93 LPA
Tables	employees, skills, employee_skills (M:N), predictions, model_runs
Key SQL	LEFT JOIN + GROUP_CONCAT; parameterised INSERT (%s)
Pre-processing	One-hot (categories), StandardScaler (numbers), 0/1 skill flags, log(salary)
Models	Linear, Ridge, Polynomial, Random Forest, **Gradient Boosting (best)**
Validation	80/20 split (960/240) + 5-fold CV; selection by CV R^2; random_state = 42
Best results	R^2 0.9566 · CV R^2 0.945 · MAE ₹1.06 L · RMSE ₹1.47 L · MAPE 8.93%
Top features	Experience 0.76 › Job role 0.65 › Location 0.11 › Education 0.08 › Company 0.04
Range	10th–90th percentile of actual/predicted ratio → 0.879 × P to 1.160 × P
Model file	models/salary_model.joblib (saved with joblib)
Version 1	cli_app.py, menu + argparse commands, ASCII charts
Version 2	web_app.py, Flask + Jinja2 + Chart.js, 5 pages, 20+ charts
Run	Windows: double-click START_HERE.bat · web at http://127.0.0.1:5000

30-second project explanation

"Our project predicts an employee's annual salary from their experience, education, skills, role, city and company type. The data is stored in MySQL in normalised tables, with skills in a many-to-many relationship. We train five regression models in scikit-learn, compare them with 5-fold cross-validation, and automatically pick the best one, which is Gradient Boosting with an R^2 of 0.957 and about 9% average error. We built two versions: a command-line app and a Flask web app with interactive charts. For each prediction the system also shows a likely range, how the person compares with peers, a growth projection, and how much each skill or degree would add to the salary."